

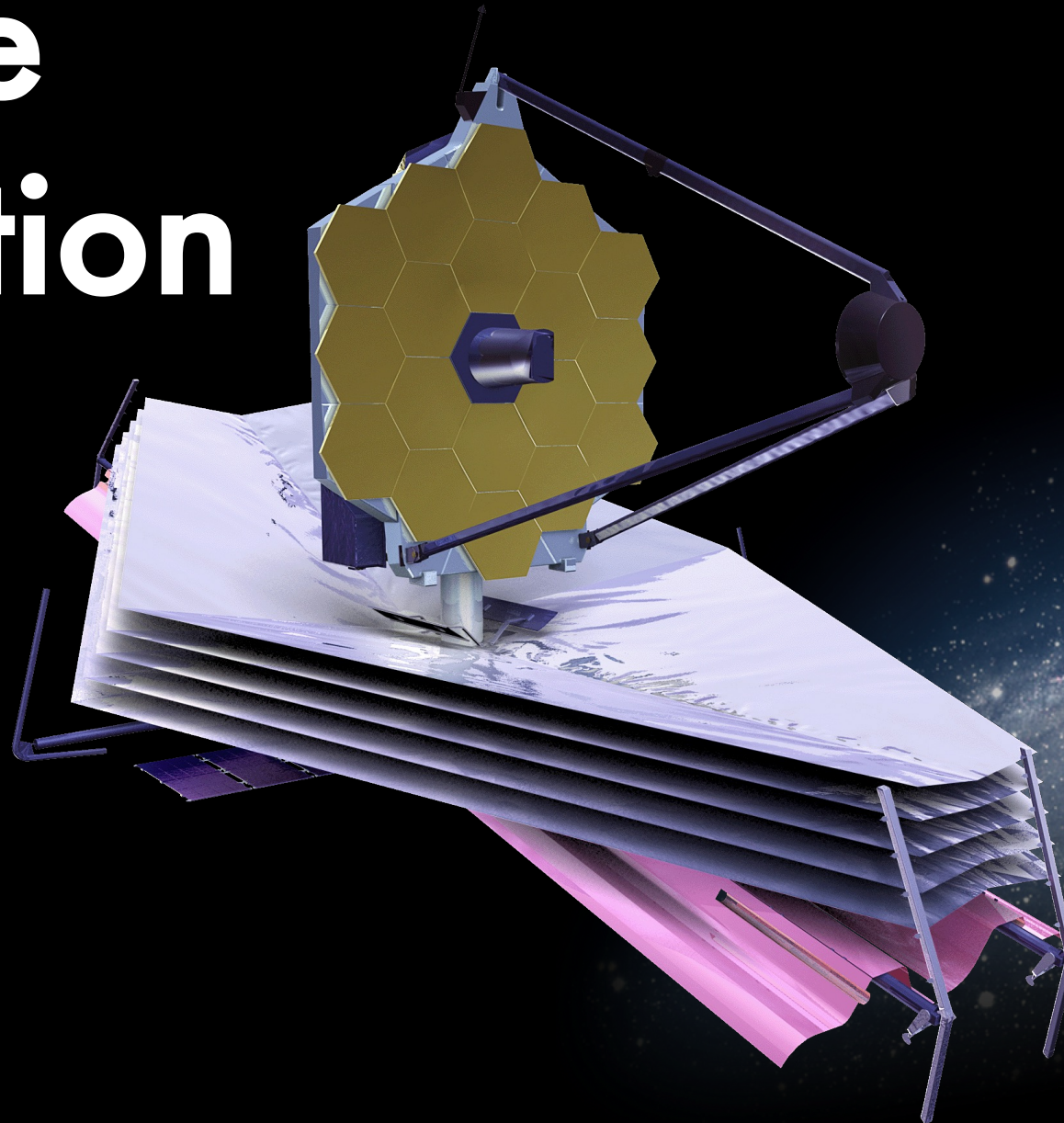
Symbolic Router Execution

Peng Zhang, Dan Wang

Aaron Gember-Jacobson

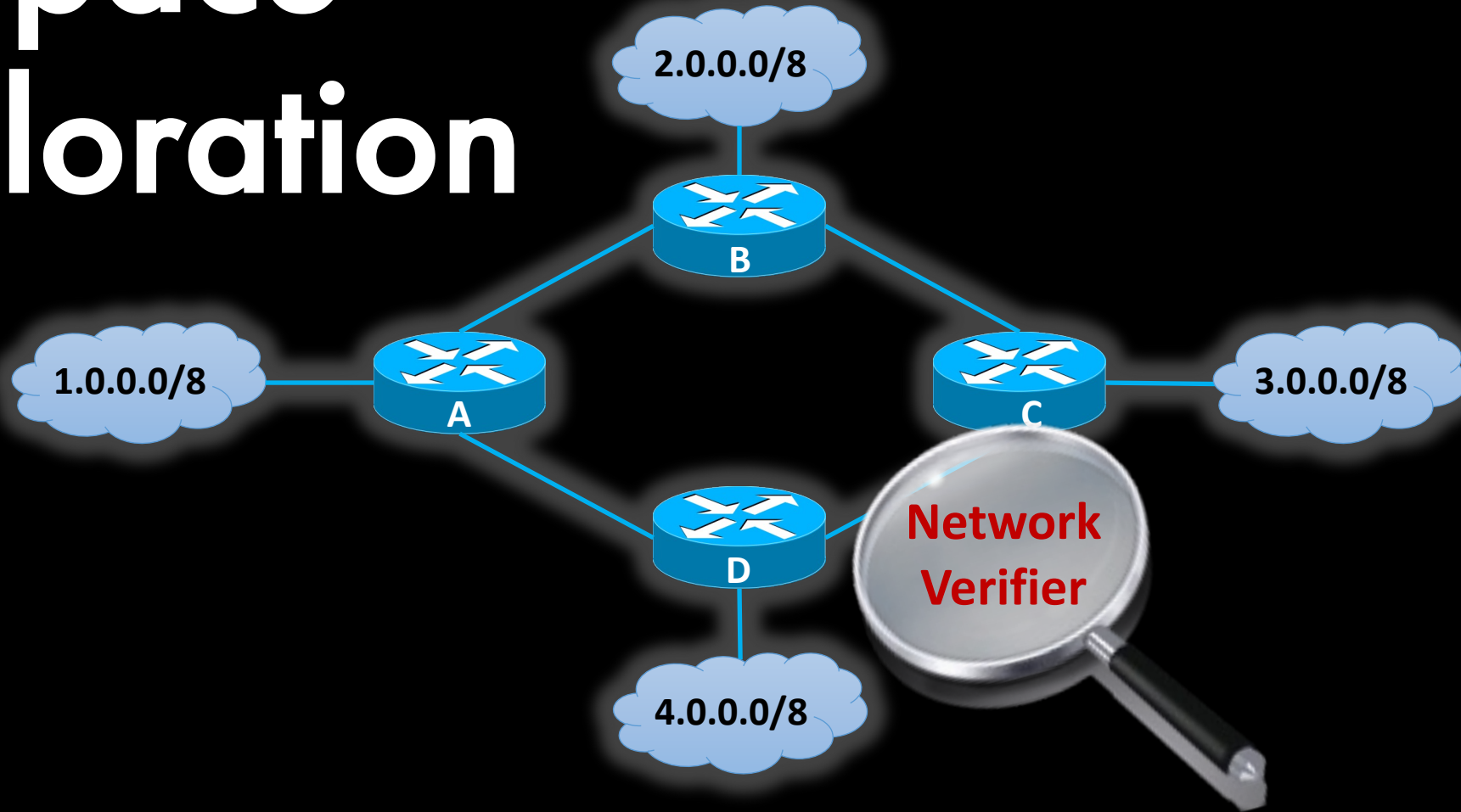


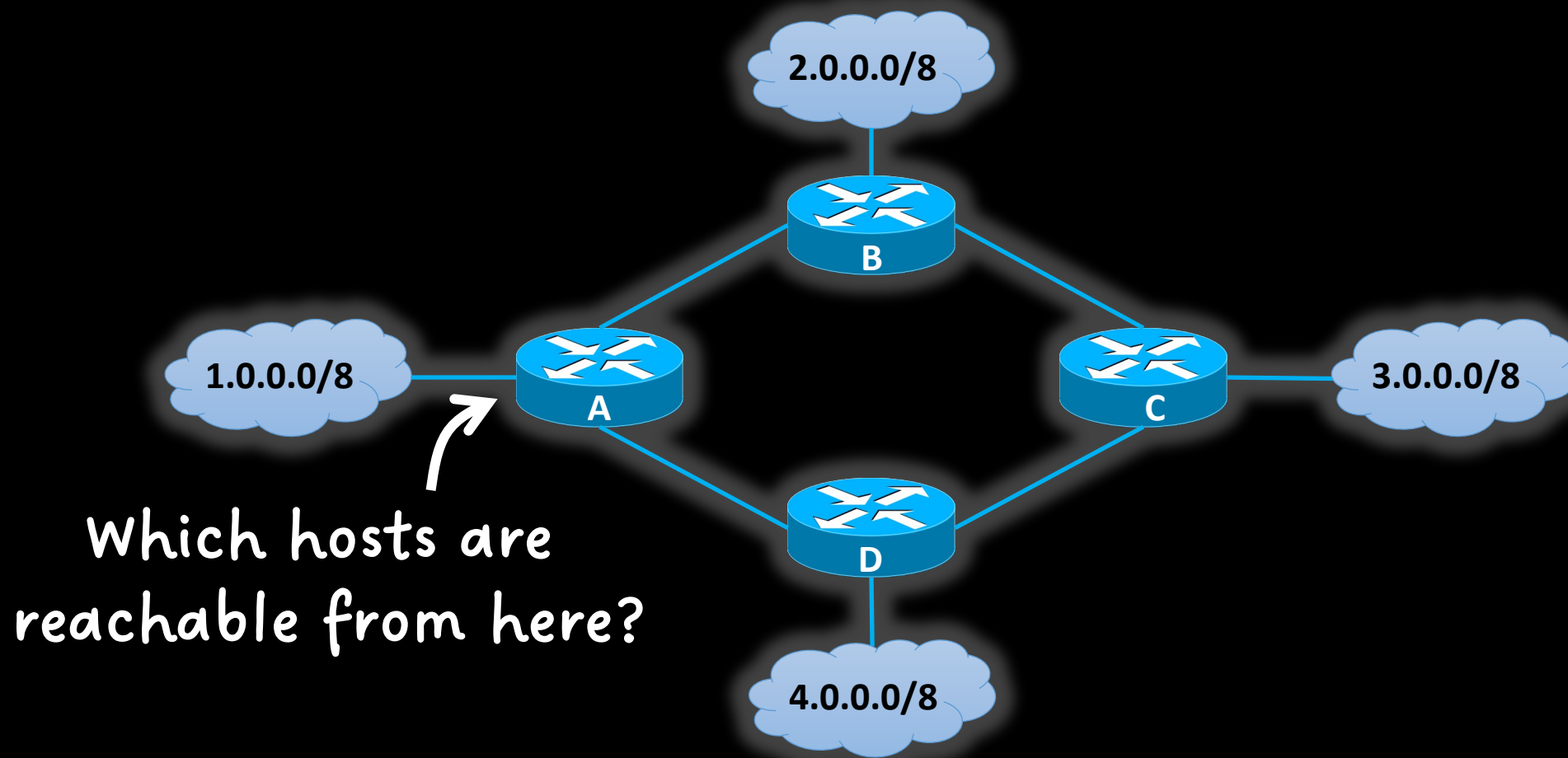
Space Exploration

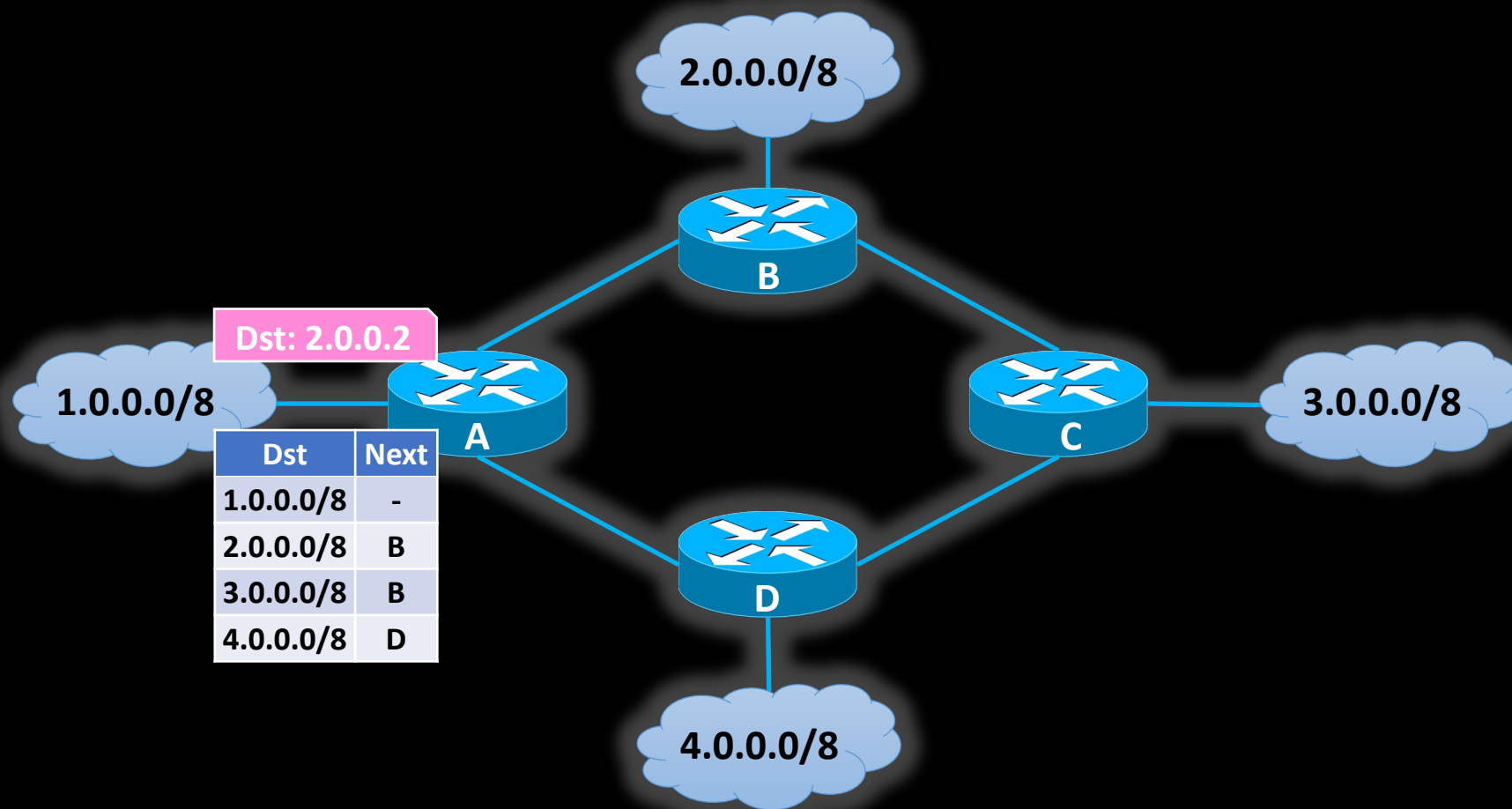


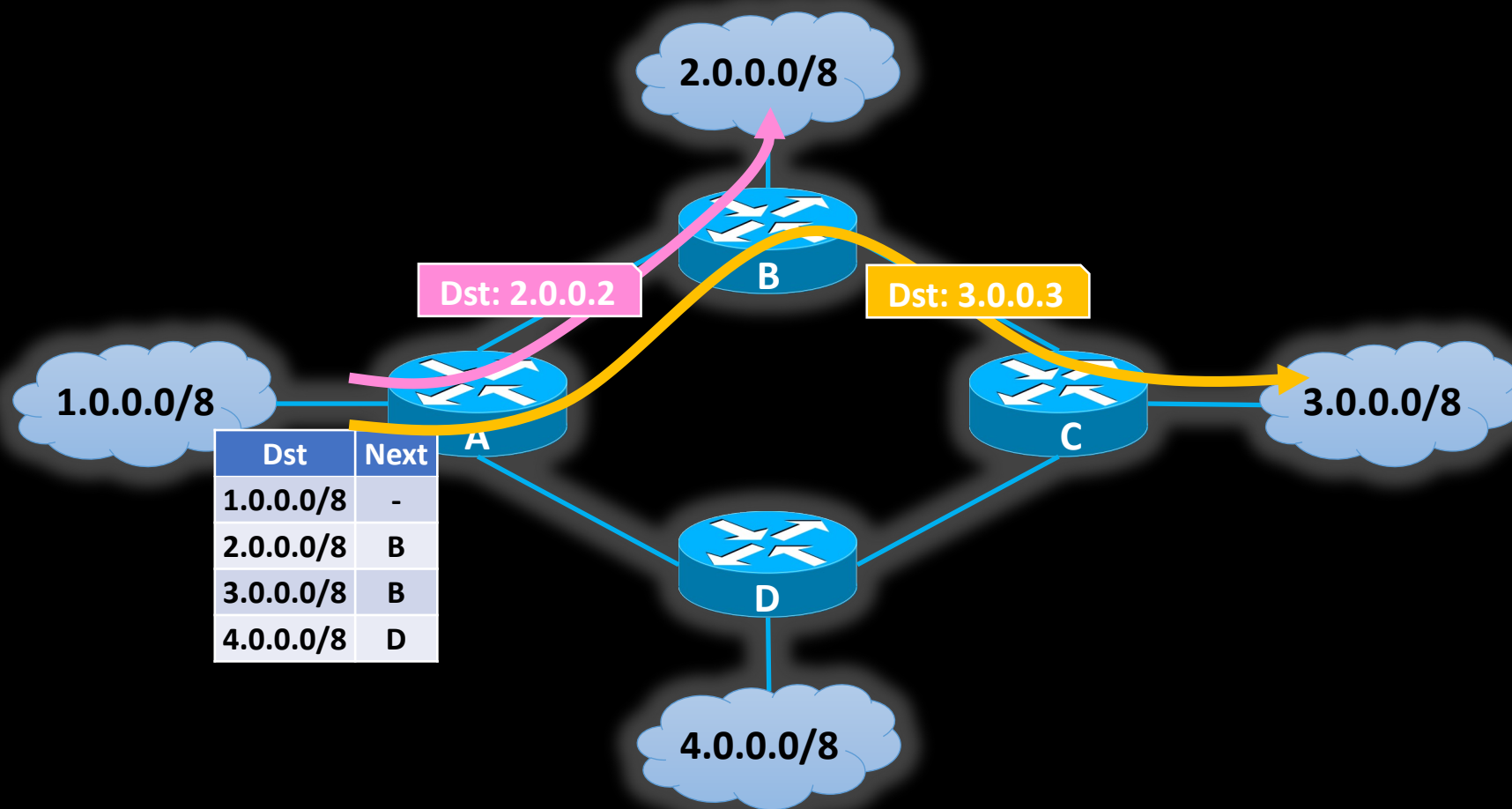
network header & failure

Space Exploration







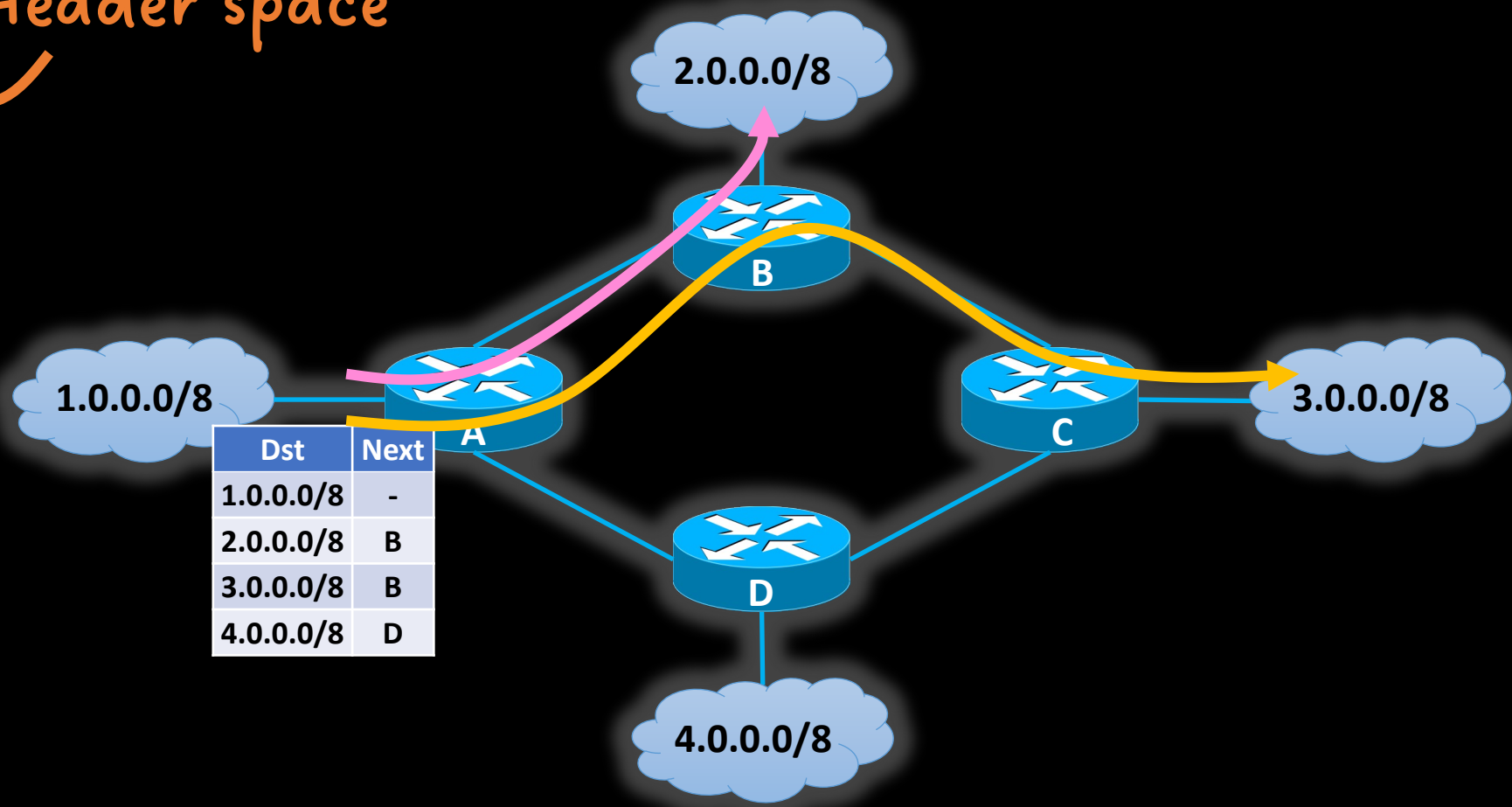




Header space



Dst	Next
1.0.0.0/8	-
2.0.0.0/8	B
3.0.0.0/8	B
4.0.0.0/8	D



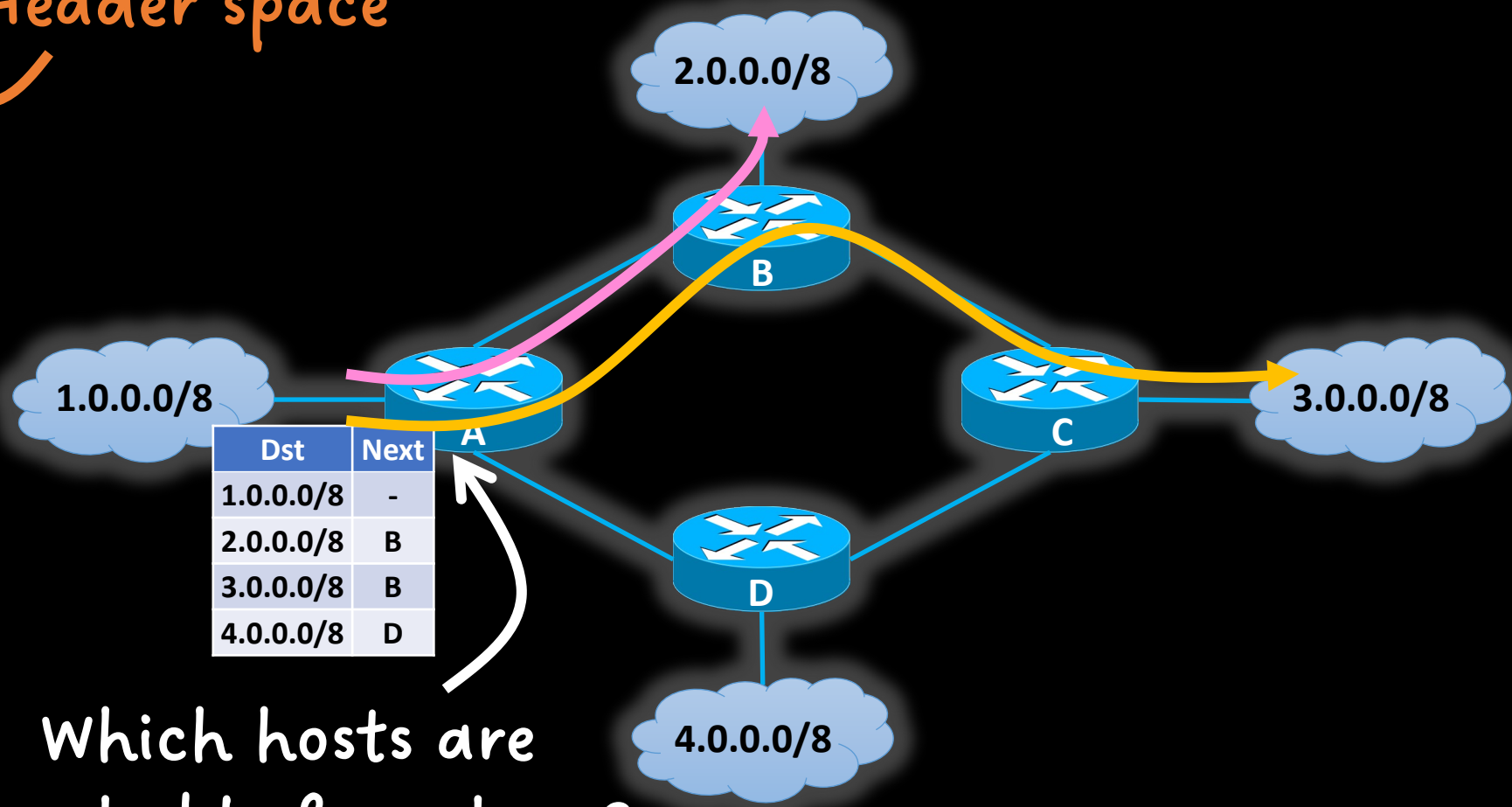


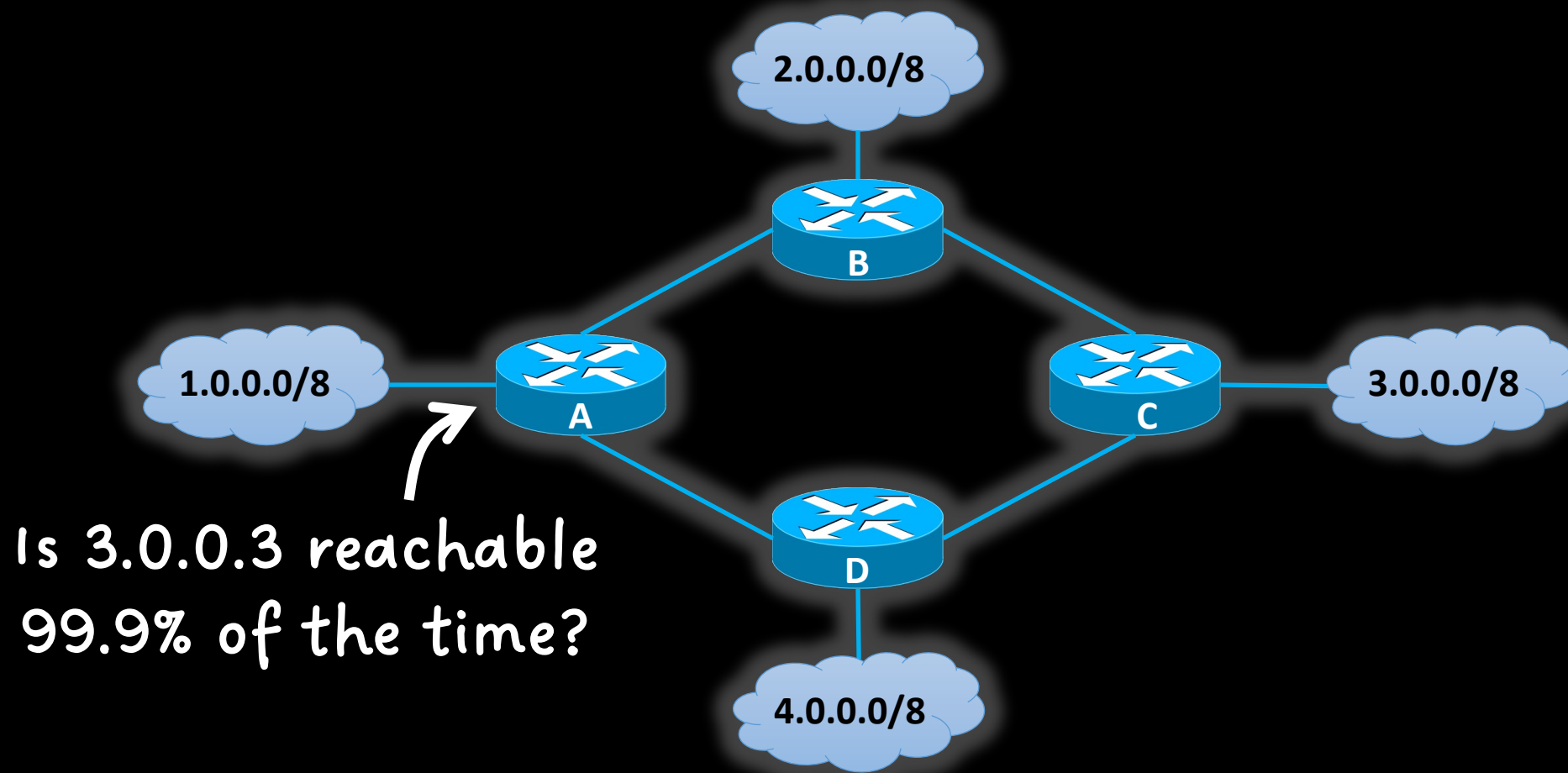
Header space

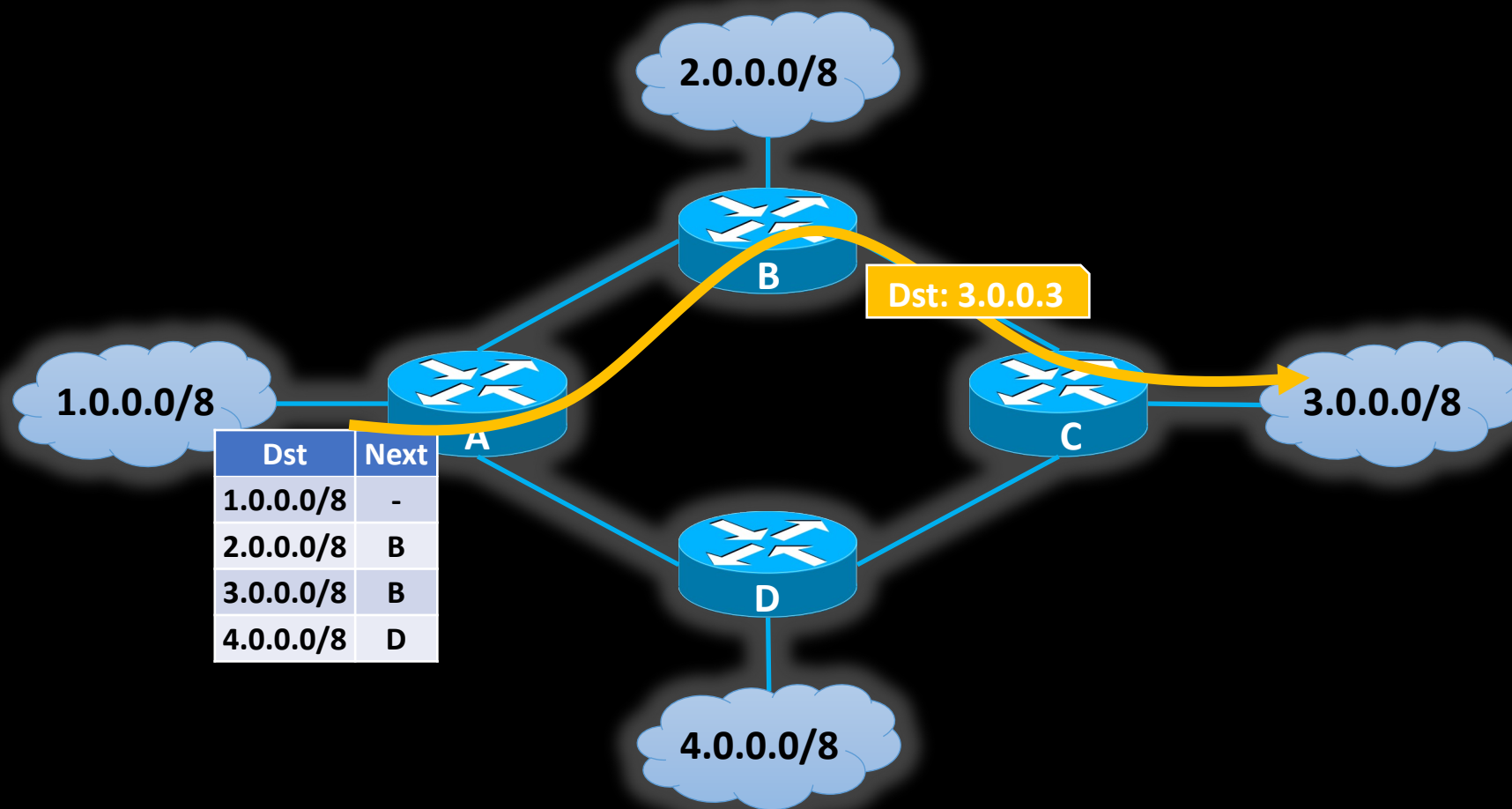


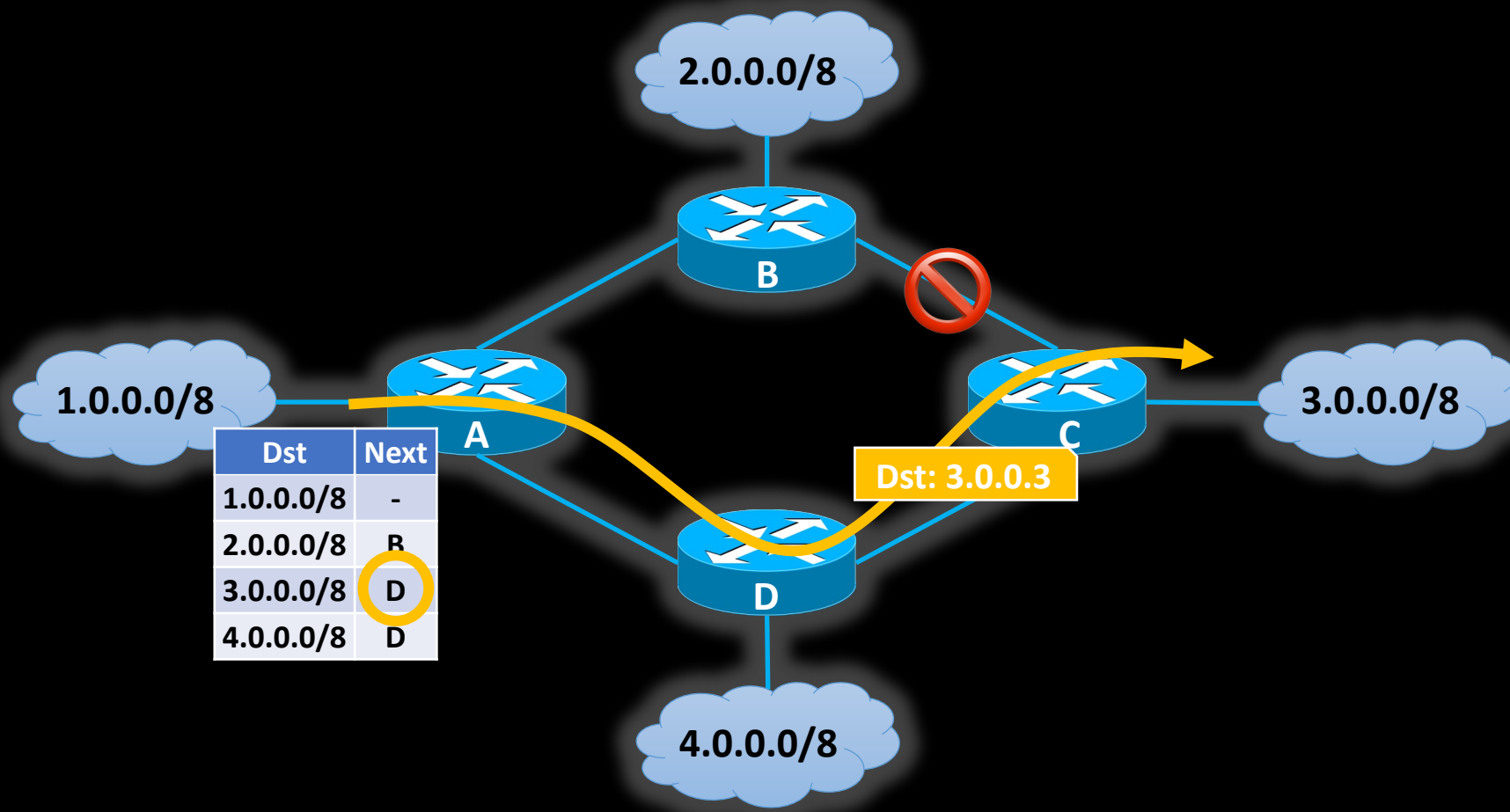
Dst	Next
1.0.0.0/8	-
2.0.0.0/8	B
3.0.0.0/8	B
4.0.0.0/8	D

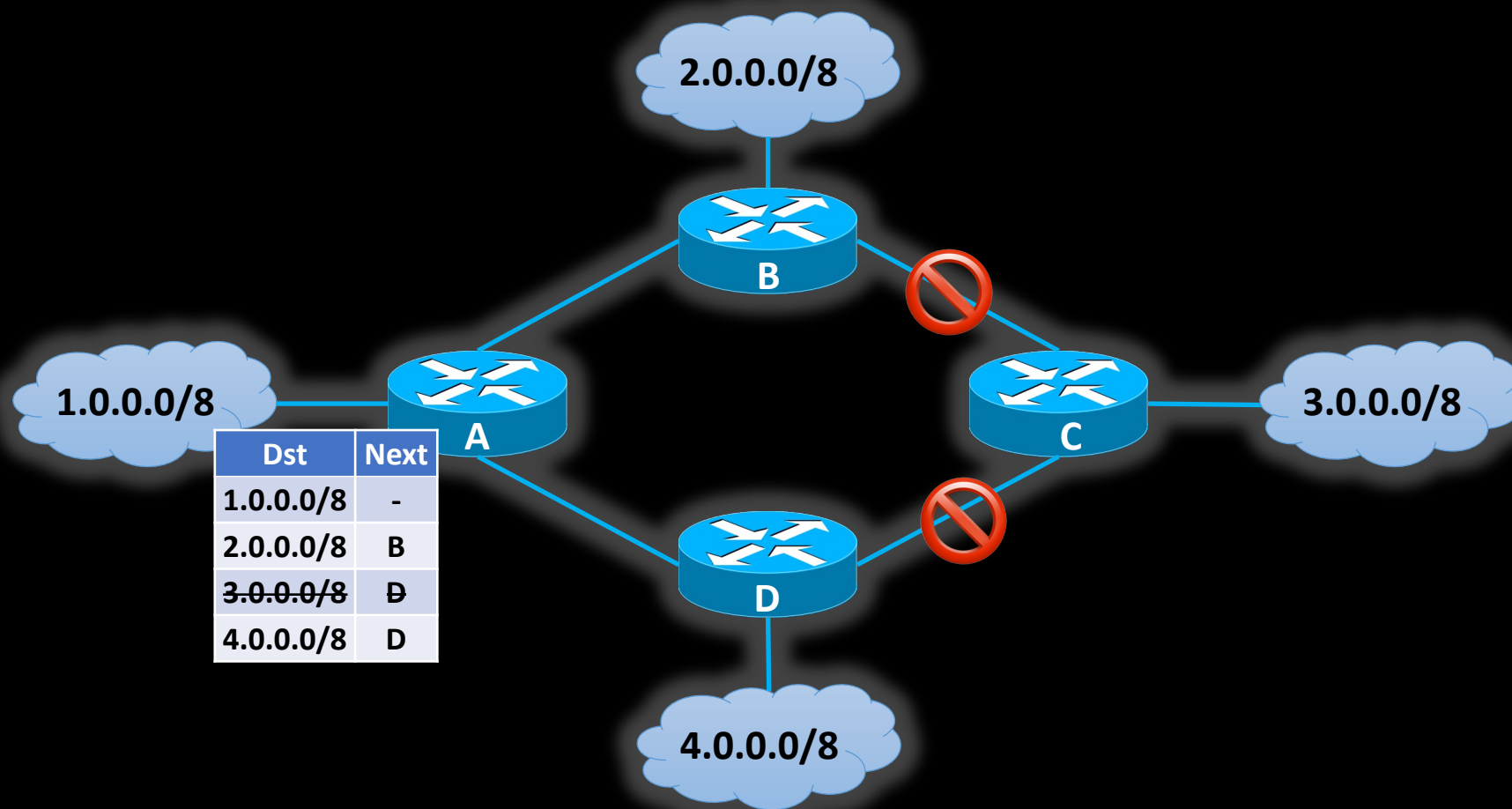
Which hosts are reachable from here?

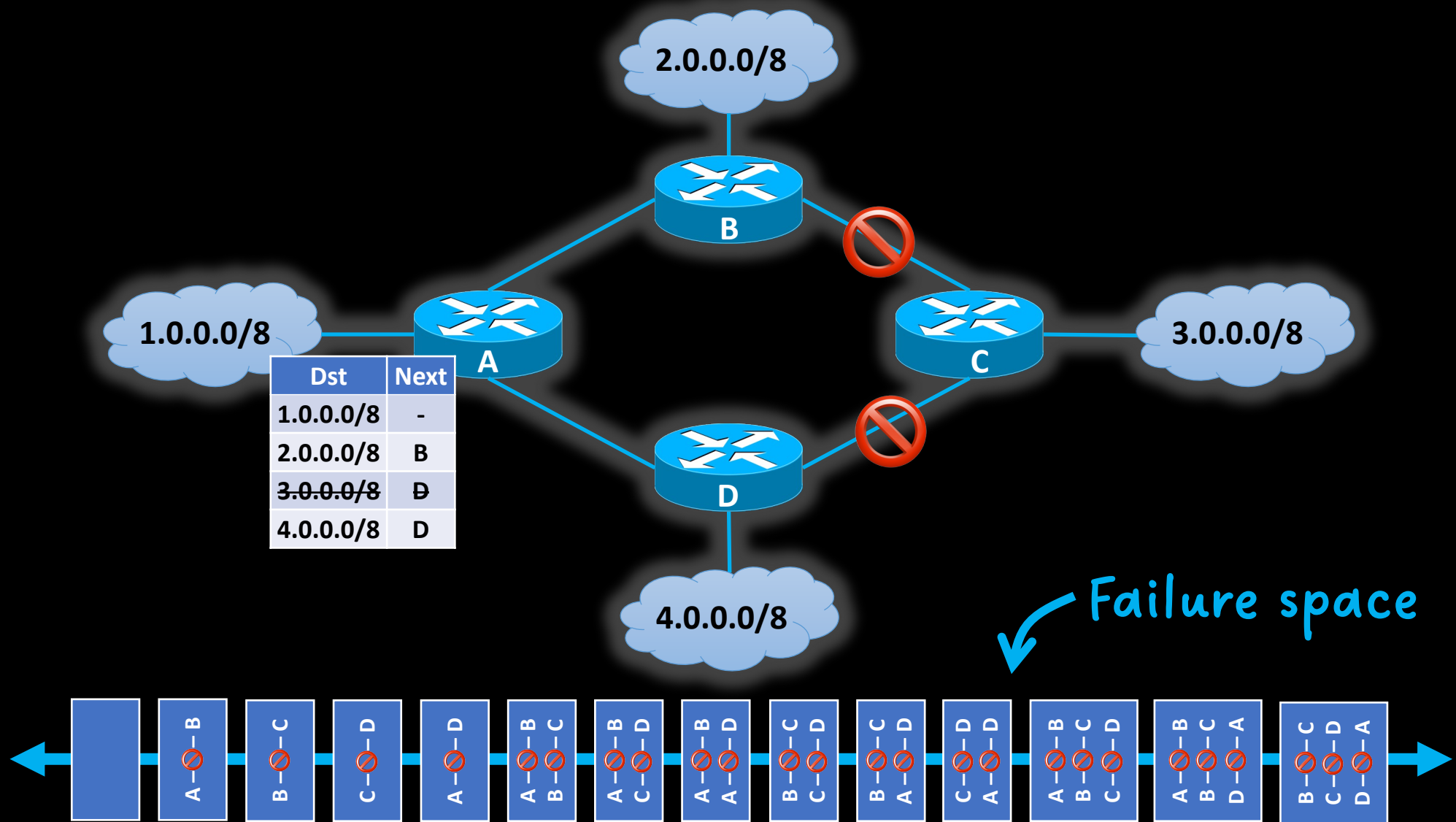




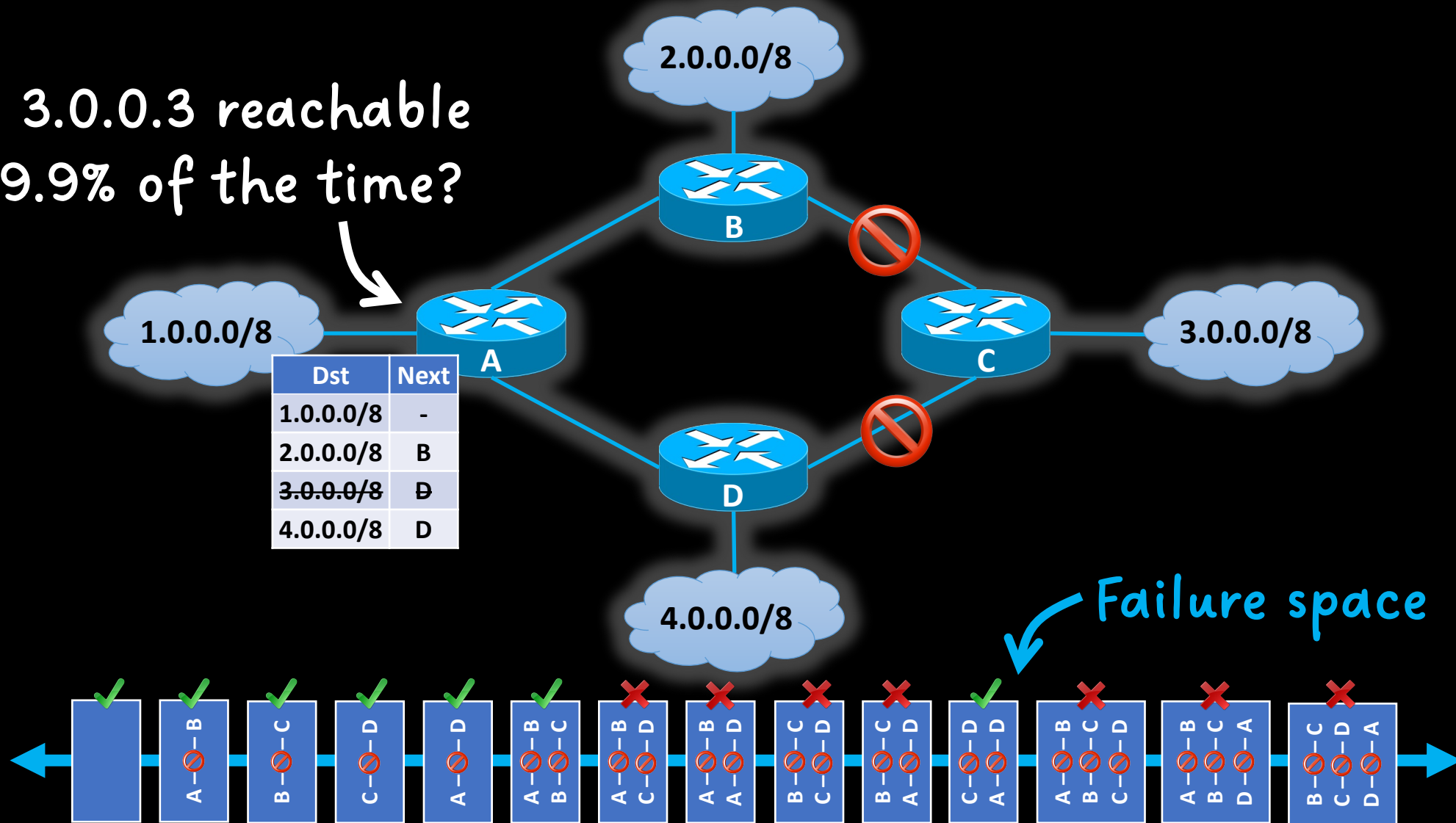




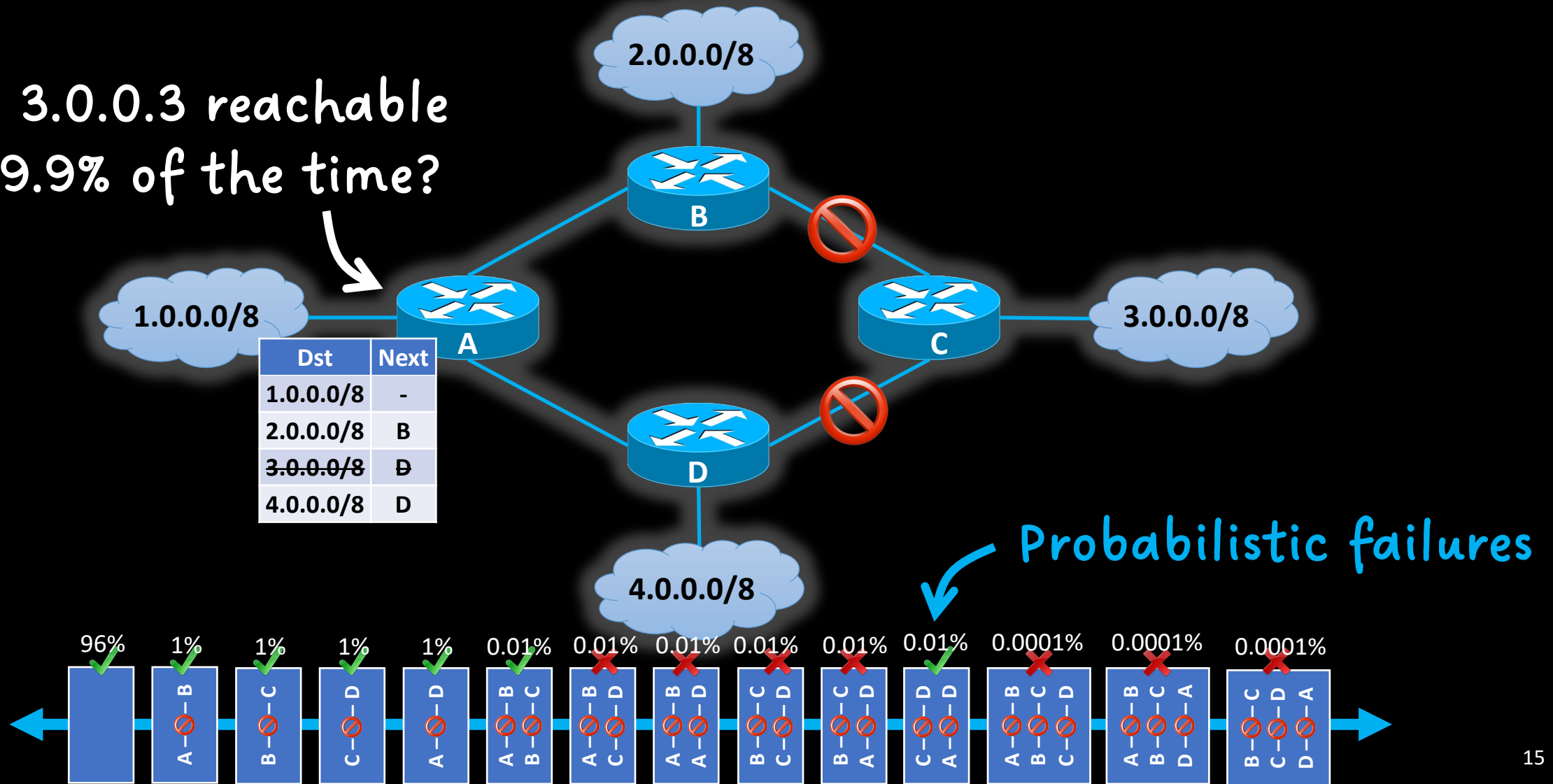




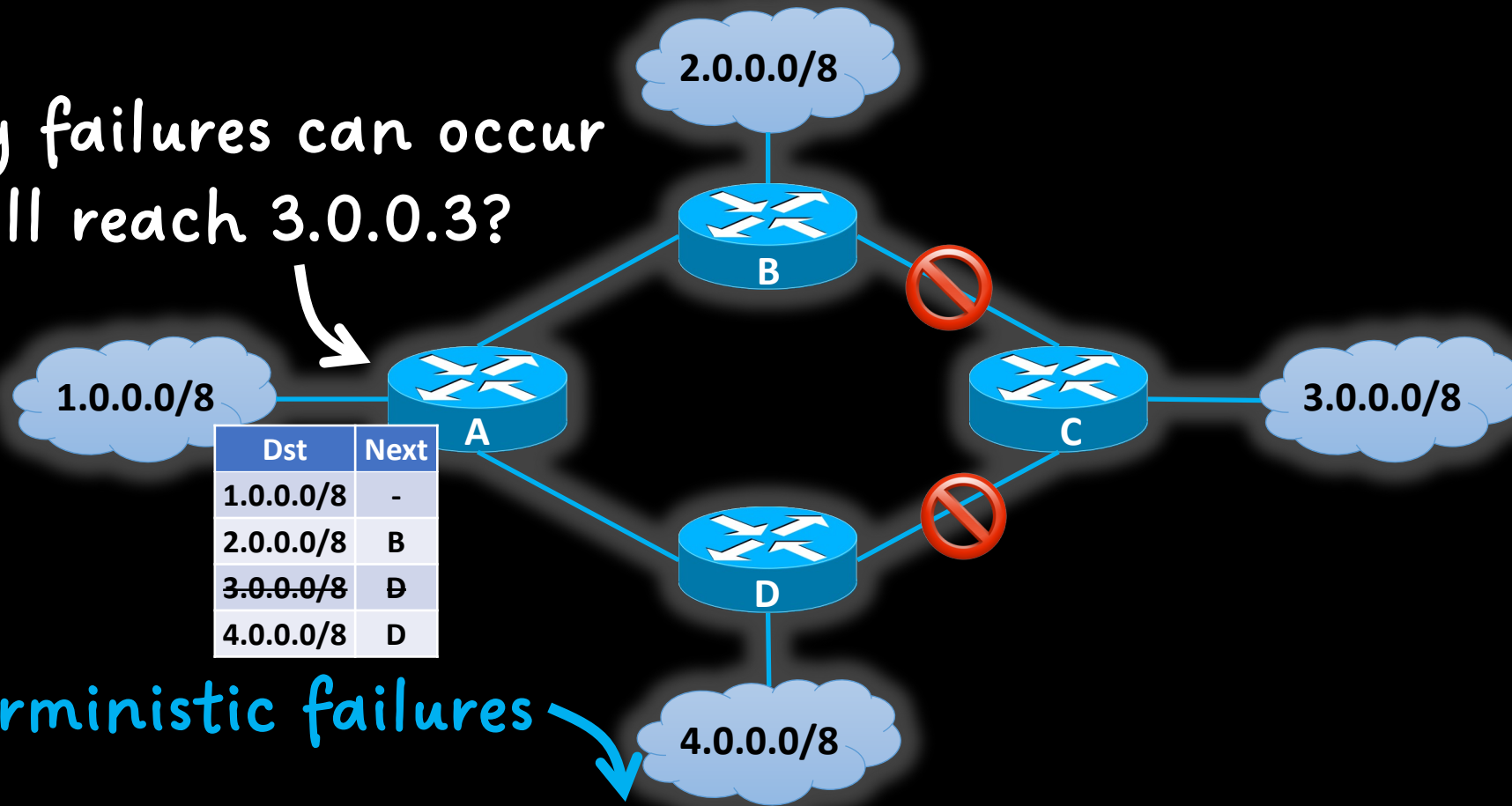
Is 3.0.0.3 reachable
99.9% of the time?



Is 3.0.0.3 reachable
99.9% of the time?

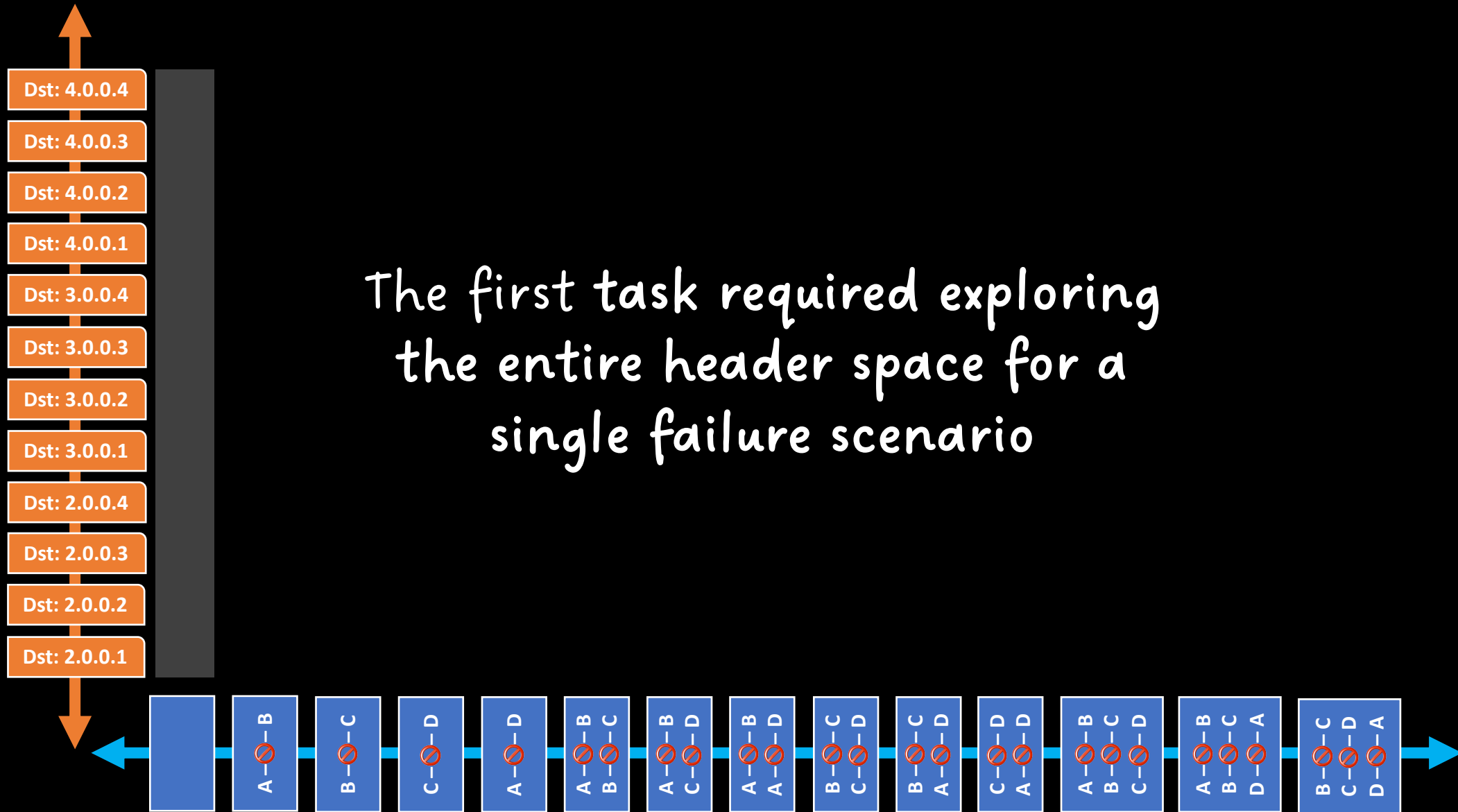


How many failures can occur and still reach 3.0.0.3?

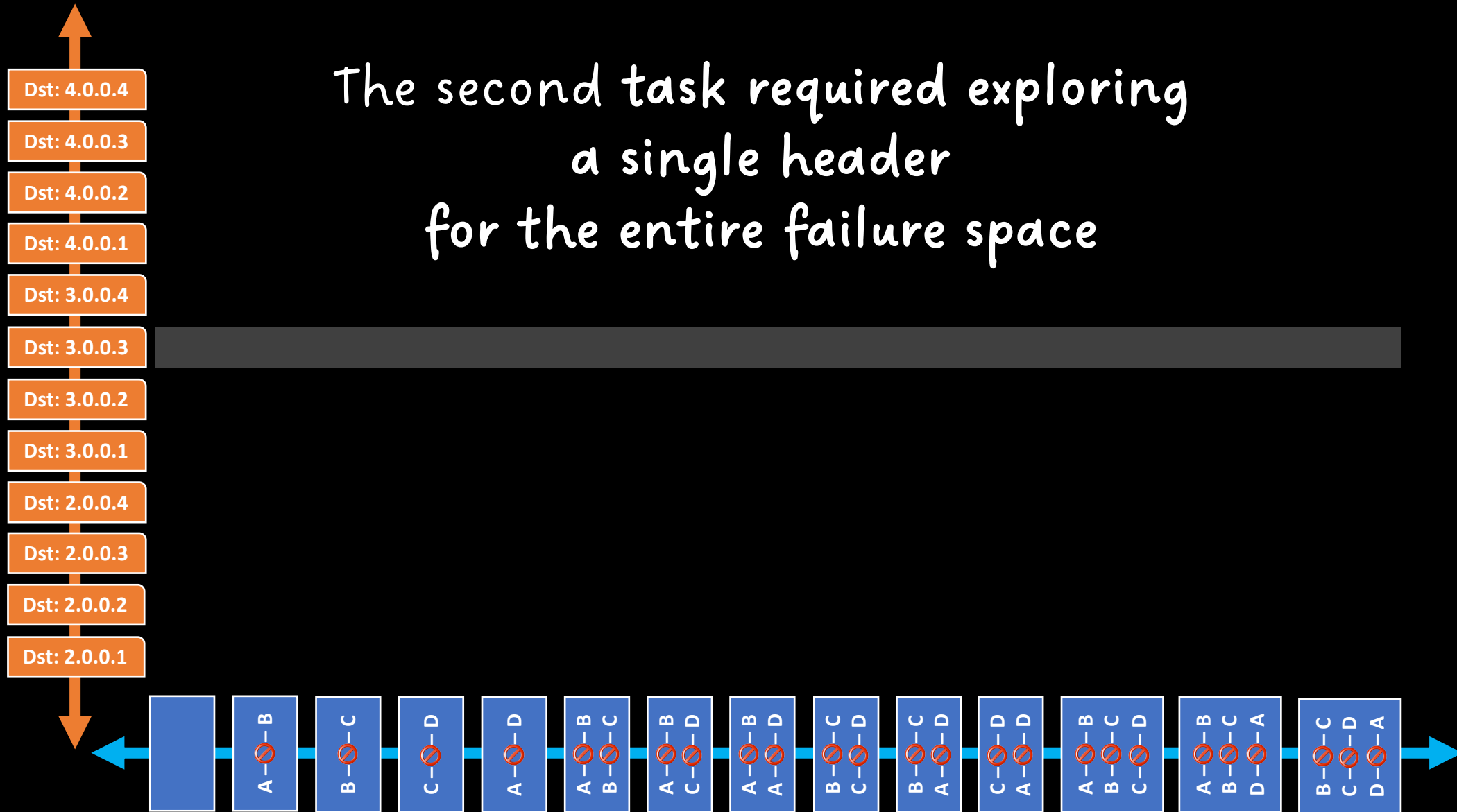


Deterministic failures



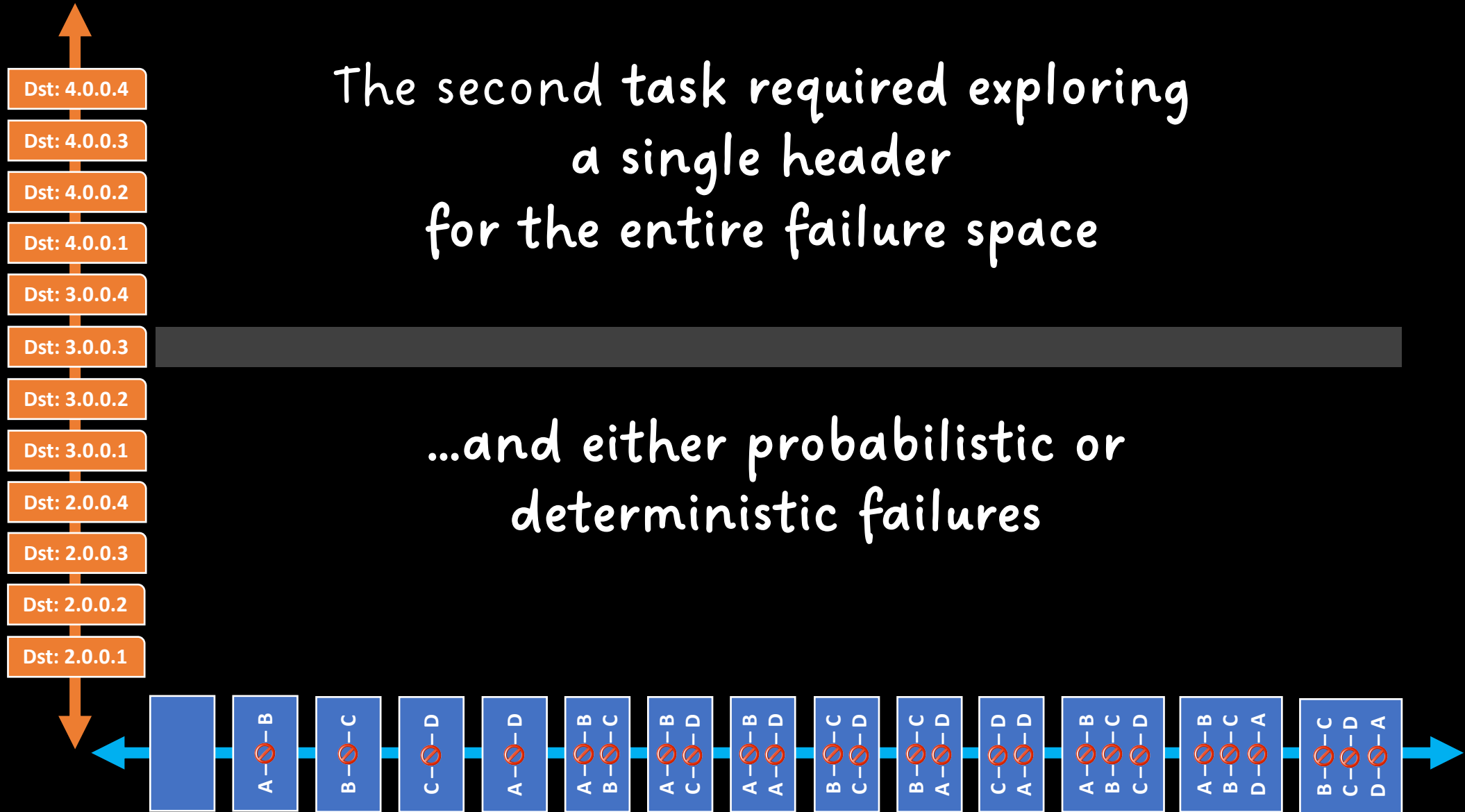


The second task required exploring
a single header
for the entire failure space

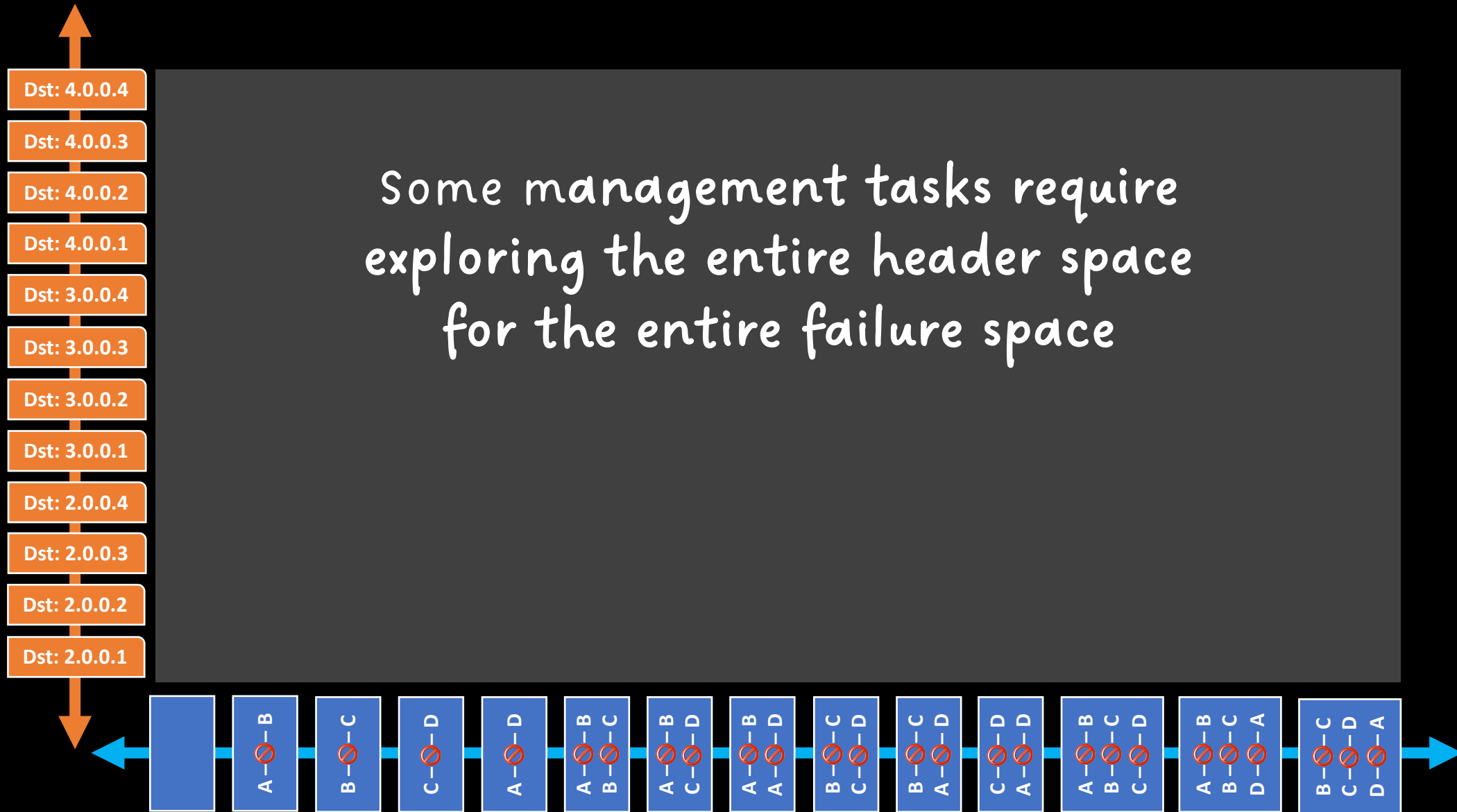


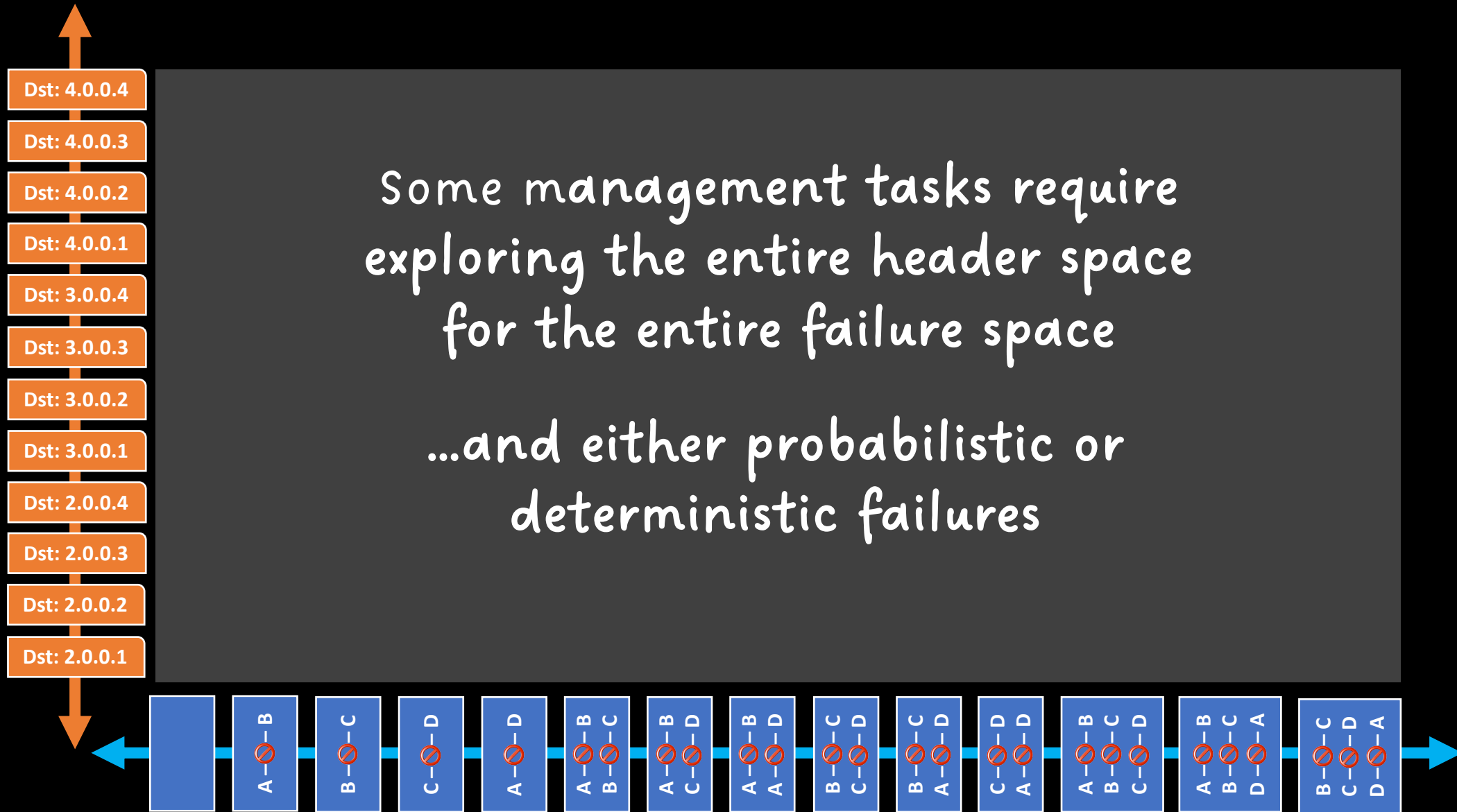
The second task required exploring
a single header
for the entire failure space

...and either probabilistic or
deterministic failures



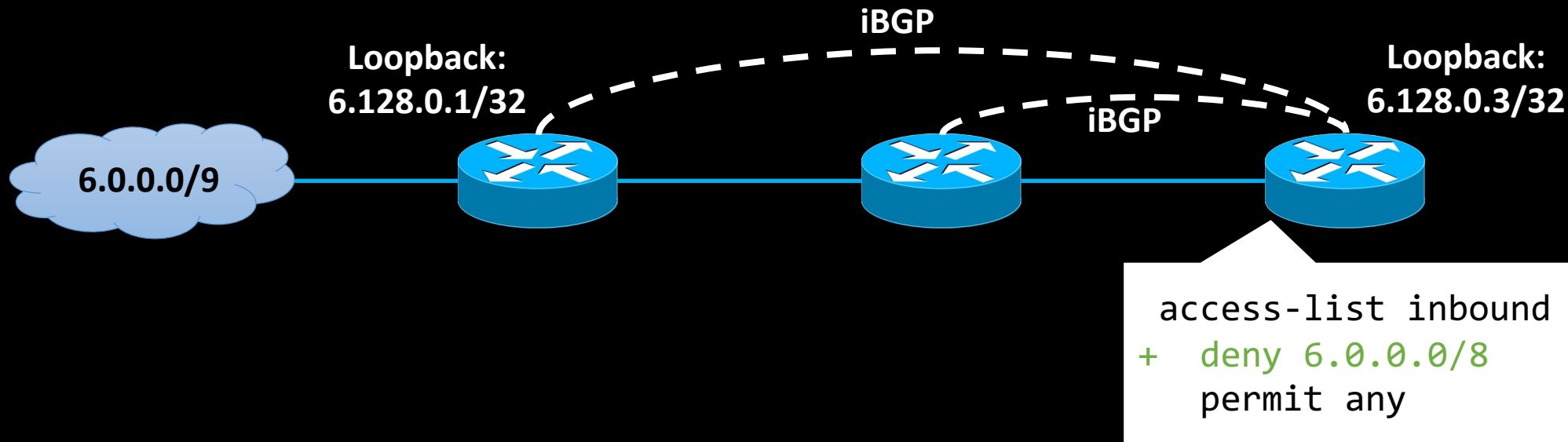
Some management tasks require
exploring the entire header space
for the entire failure space





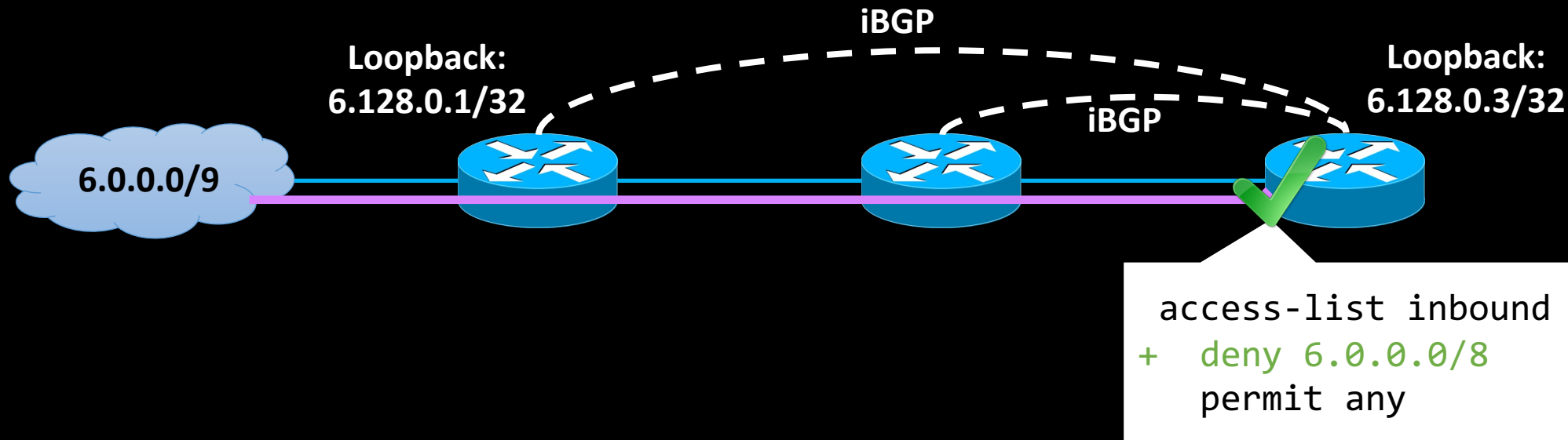
Some management tasks require exploring the product space

① Verify a proposed configuration change has no side-effects



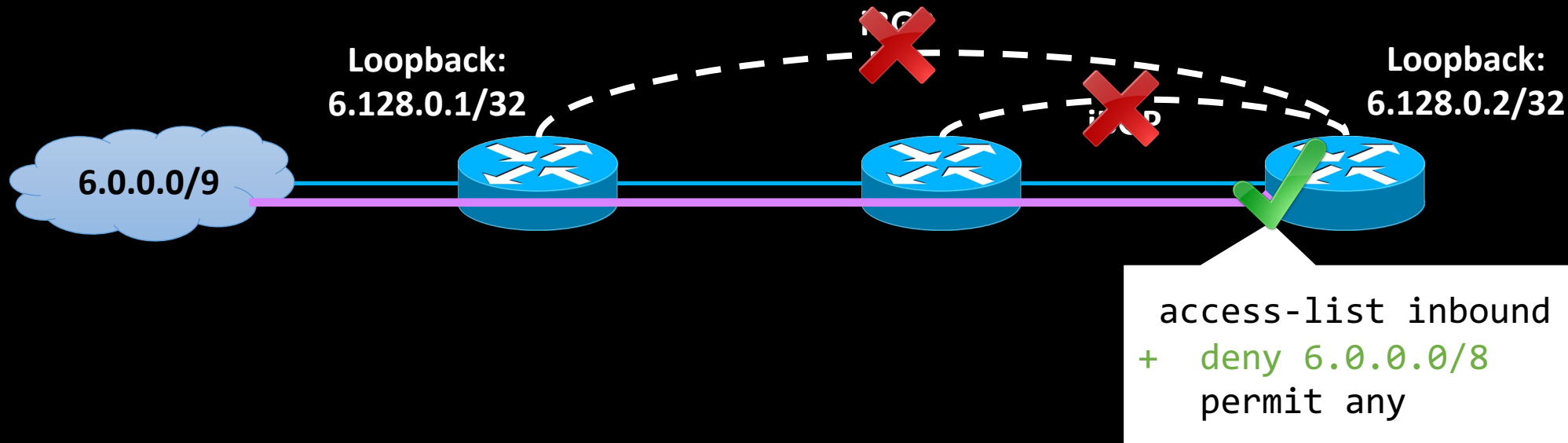
Some management tasks require exploring the product space

① Verify a proposed configuration change has no side-effects



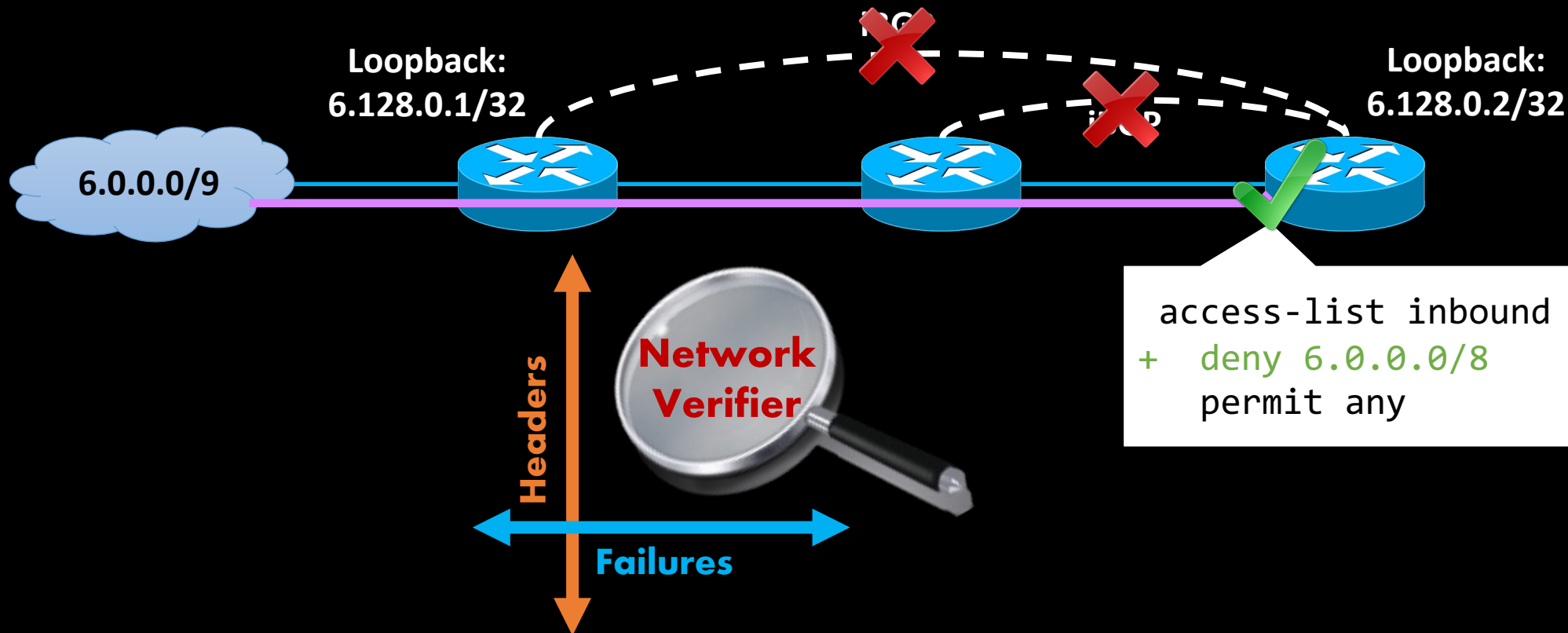
Some management tasks require exploring the product space

① Verify a proposed configuration change has no side-effects



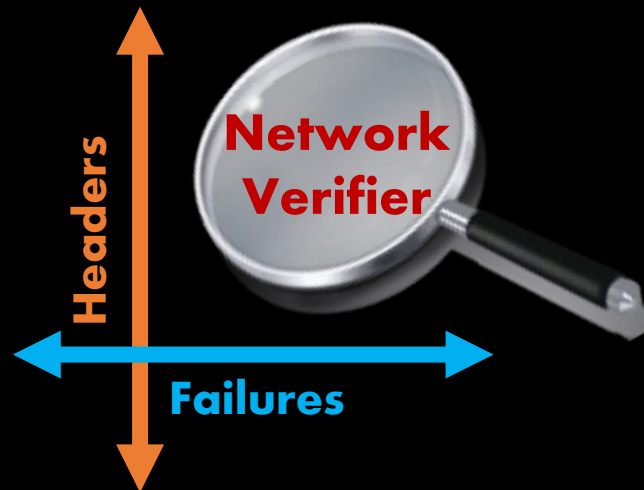
Some management tasks require exploring the product space

① Verify a proposed configuration change has no side-effects



Some management tasks require exploring the product space

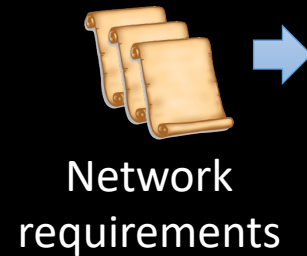
- ① Verify a proposed configuration change has no side-effects
- ② Mine network requirements (e.g., Config2Spec)



Some management tasks require exploring the product space

① Verify a proposed configuration change has no side-effects

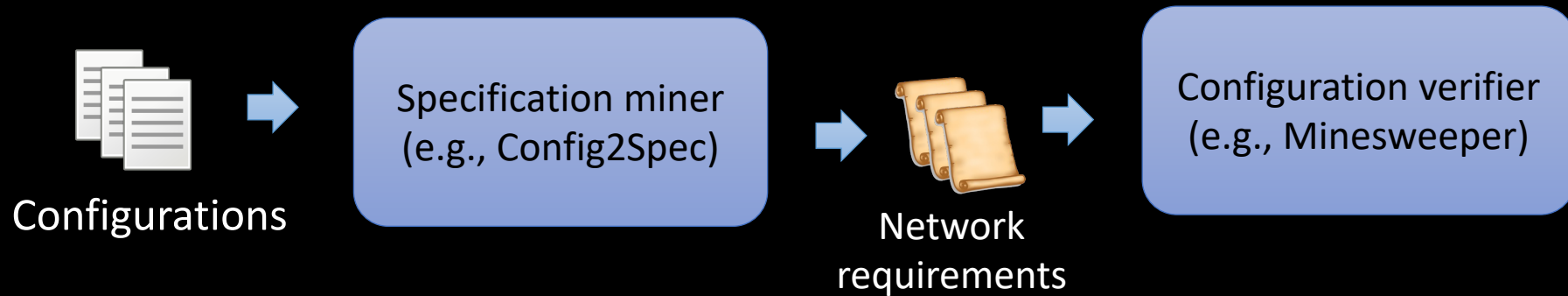
② Mine network requirements (e.g., Config2Spec)



Some management tasks require exploring the product space

① Verify a proposed configuration change has no side-effects

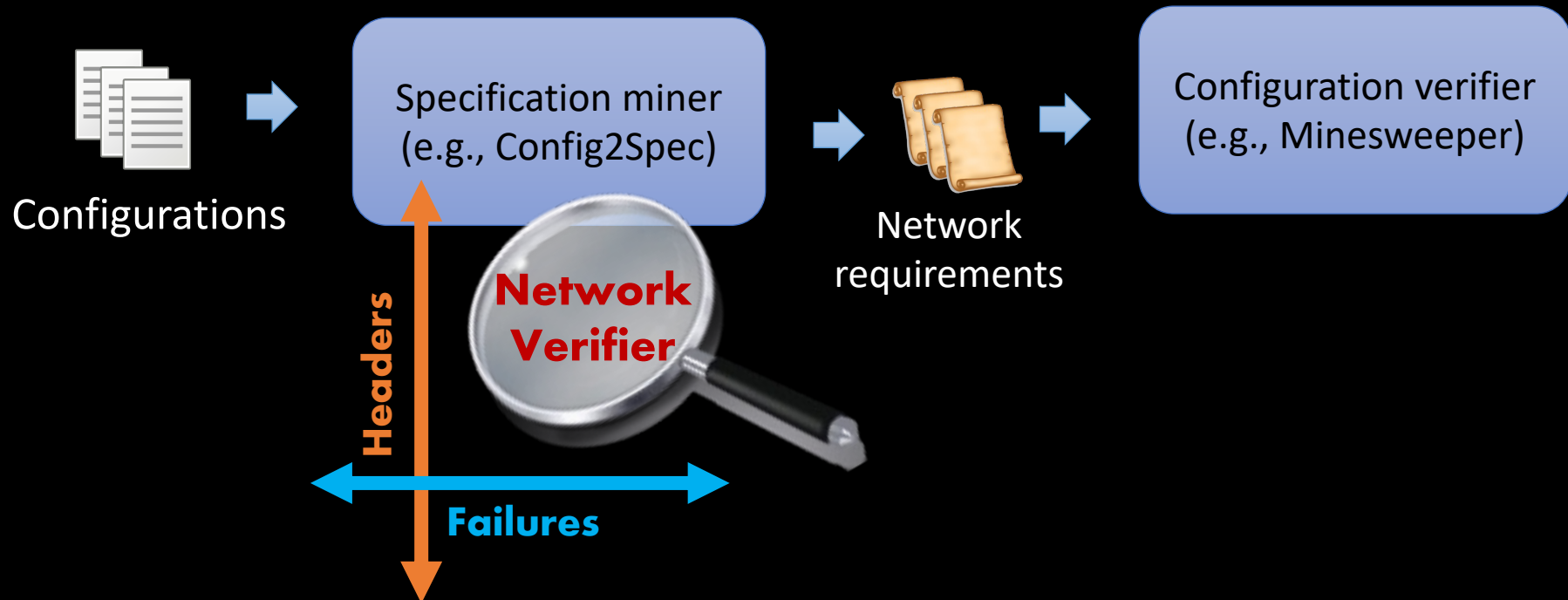
② Mine network requirements (e.g., Config2Spec)

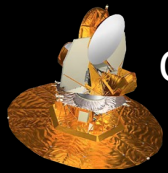


Some management tasks require exploring the product space

① Verify a proposed configuration change has no side-effects

② Mine network requirements (e.g., Config2Spec)

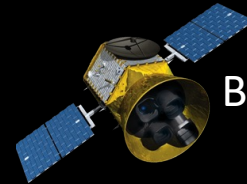




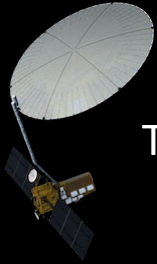
Config2Spec



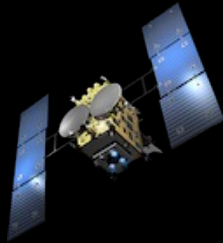
ARC



Bagpipe



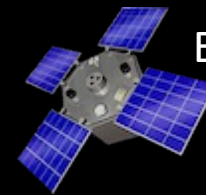
Tiramisu



Hoyan



Plankton



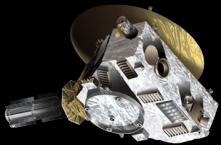
ERA



ShapeShifter



Minesweeper



Bagpipe



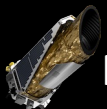
Origami



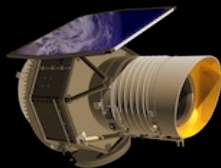
DNA



NV

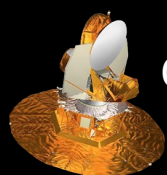


NetDice



ProbNV

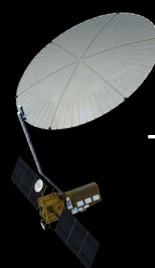
Can we use
the verifiers we
already have?



Config2Spec



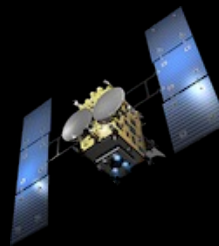
ARC



Tiramisu



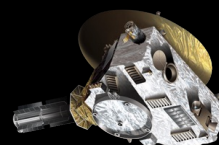
Minesweeper



Hoyan



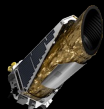
Origami



Bagpipe



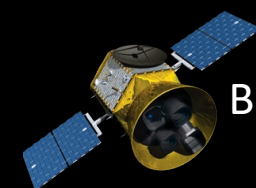
NV



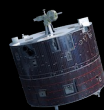
NetDice



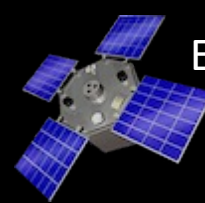
ProbNV



Bagpipe



Plankton



ERA



ShapeShifter

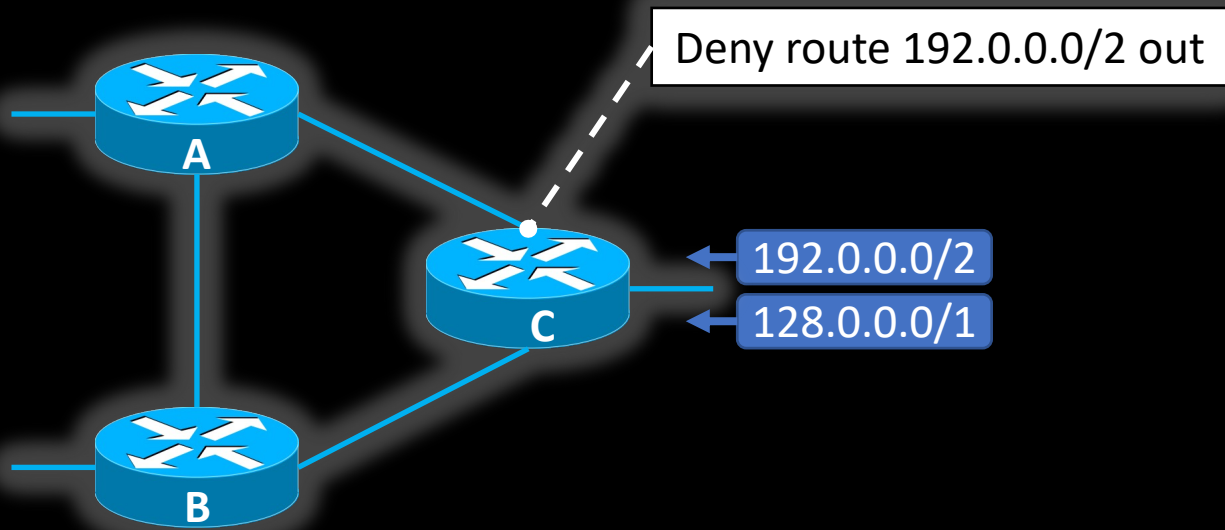


DNA

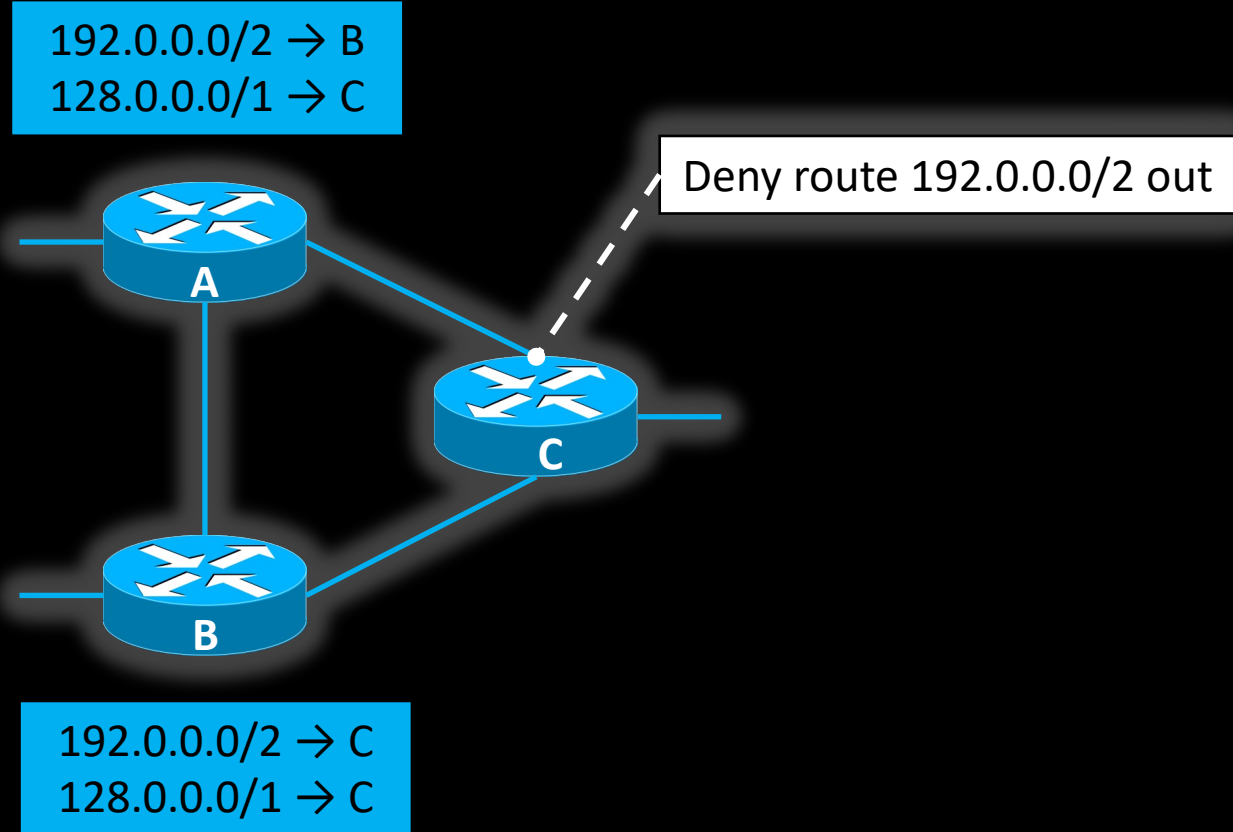
Divide headers into PECs

Efficient header
space exploration

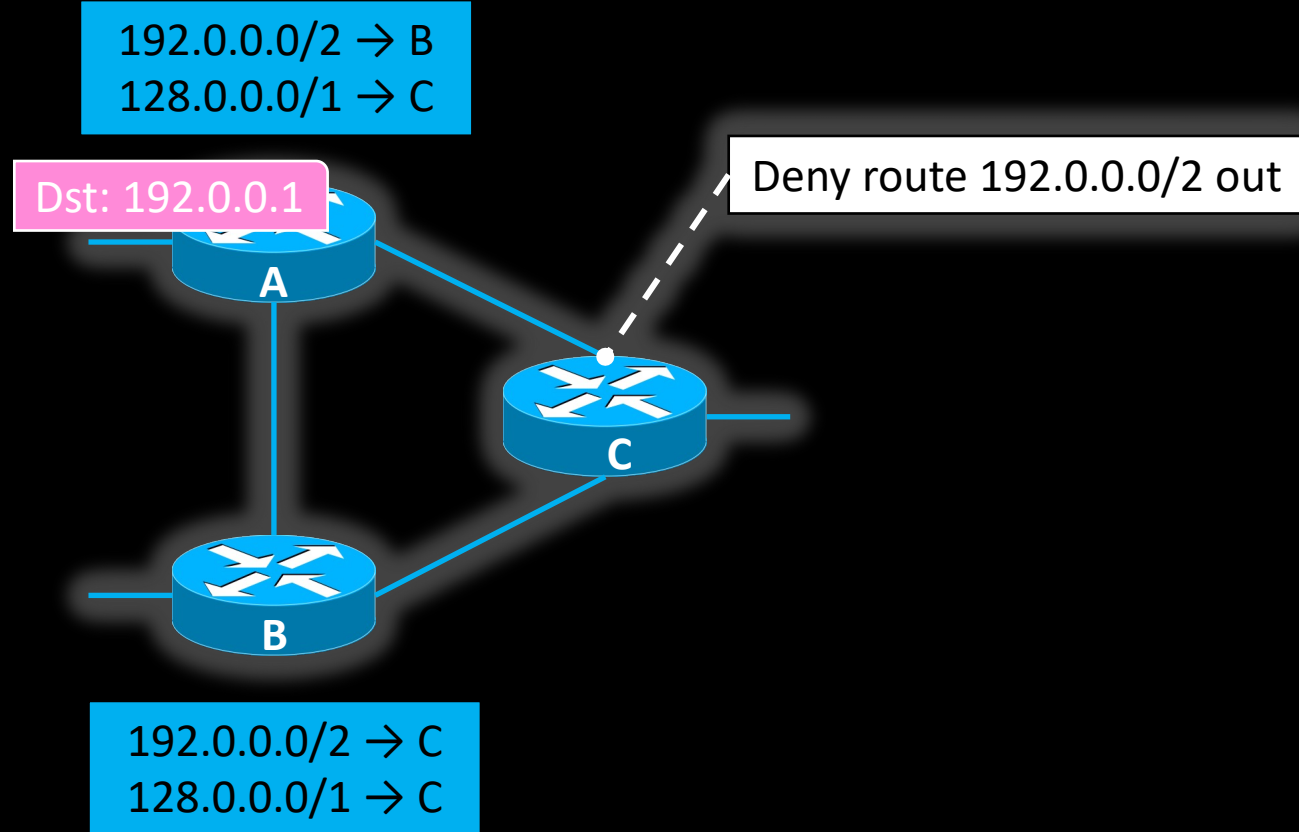
Packet Equivalence Classes



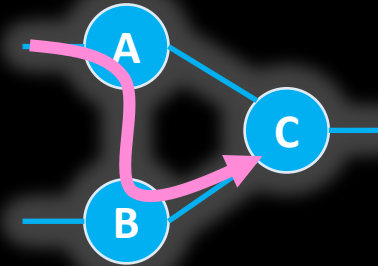
Packet Equivalence Classes



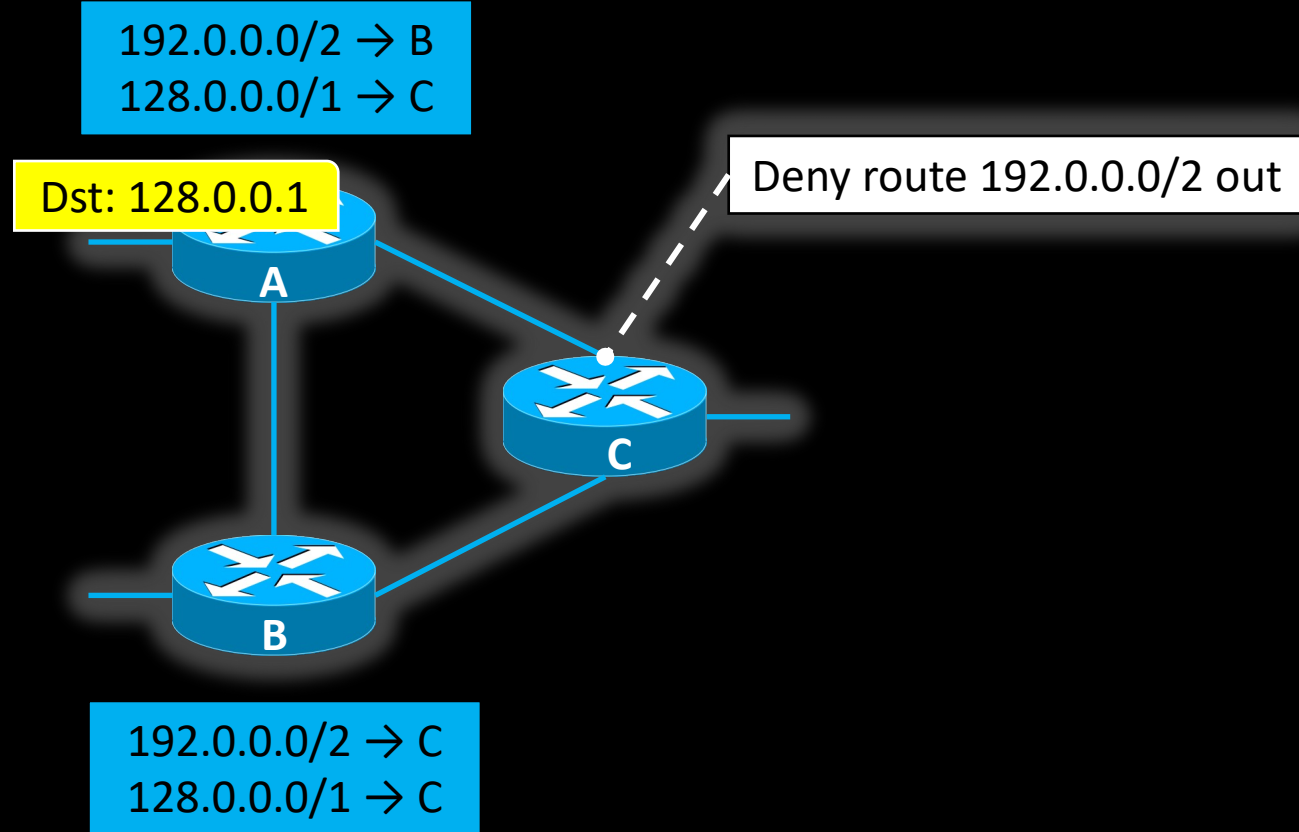
Packet Equivalence Classes



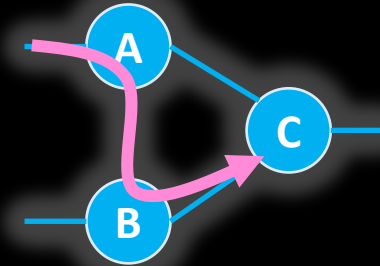
192.0.0.0/2



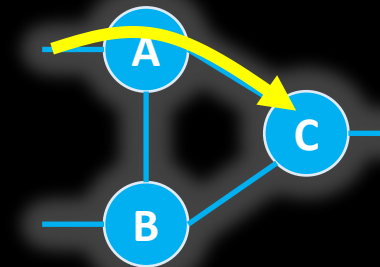
Packet Equivalence Classes



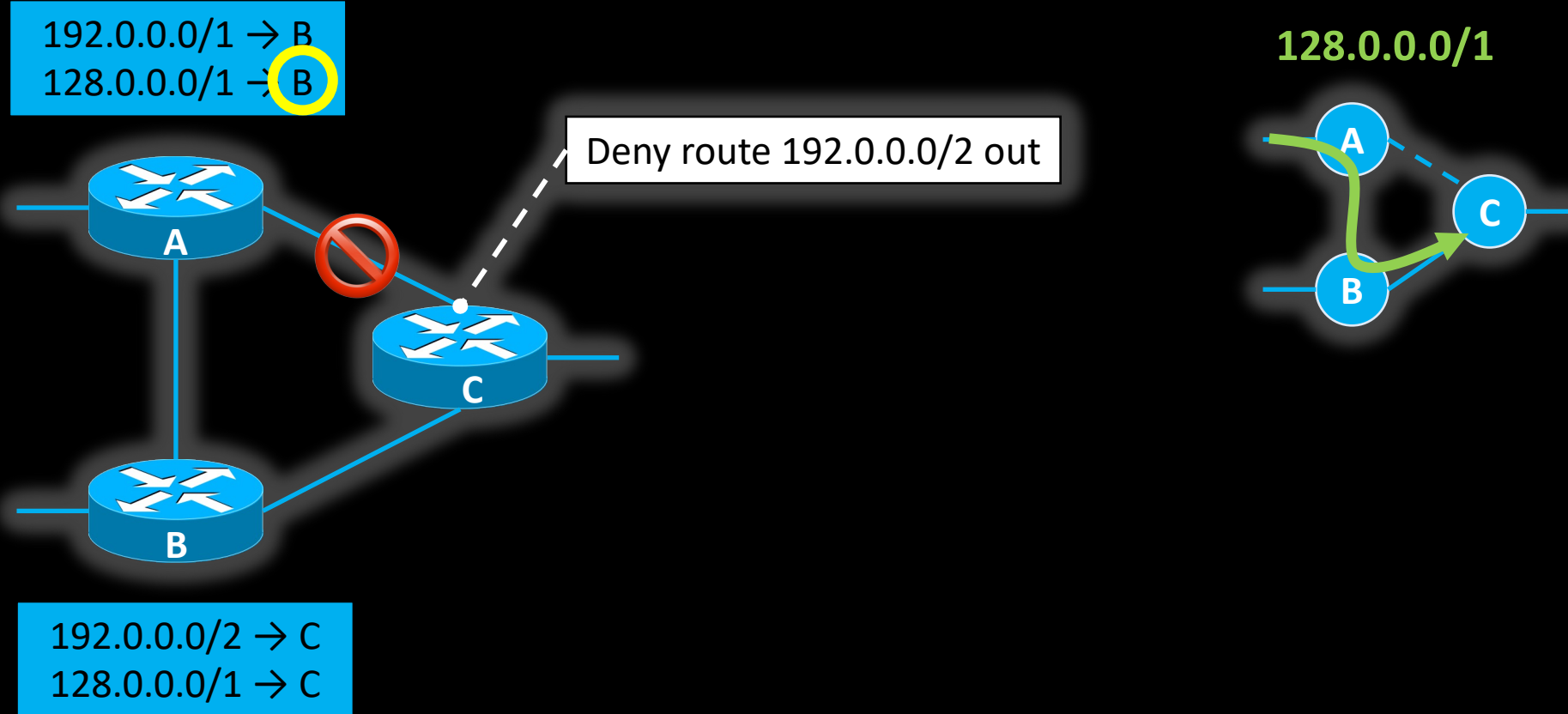
192.0.0.0/2

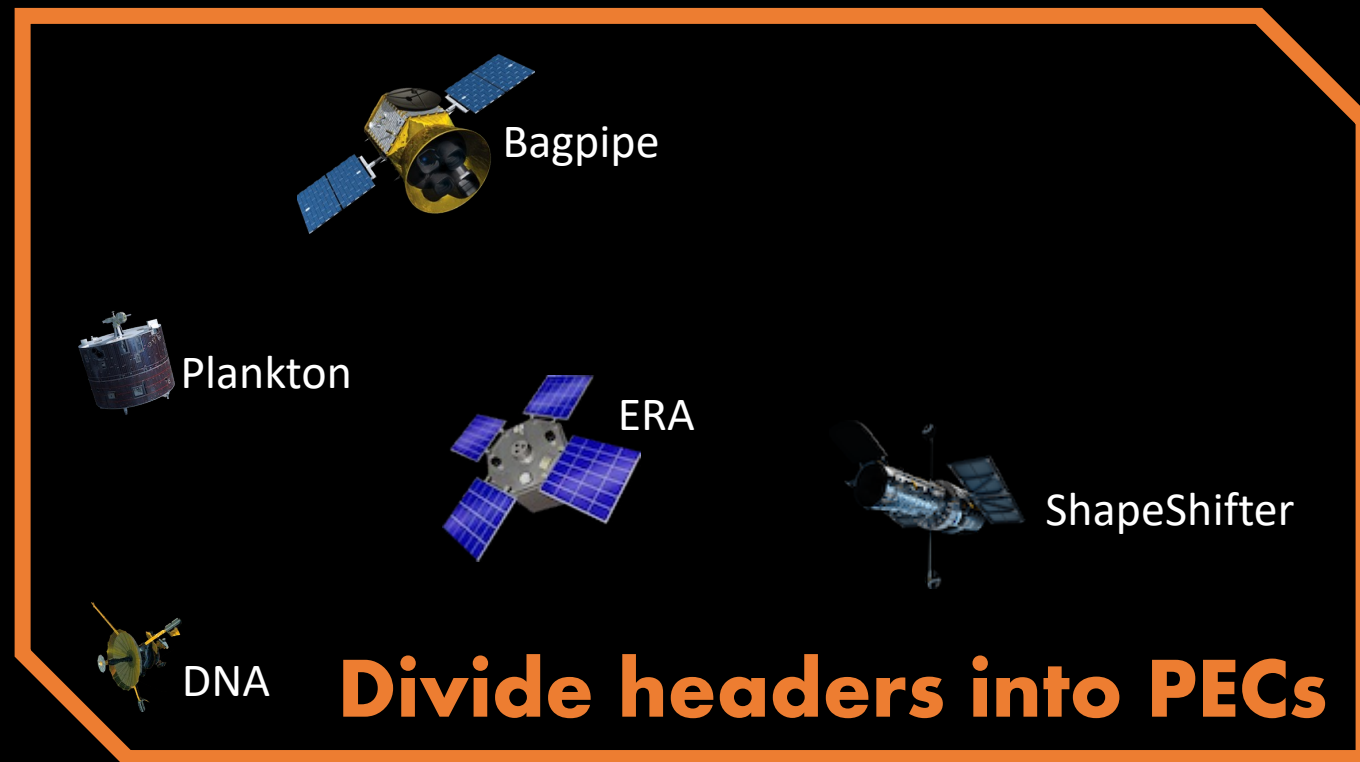
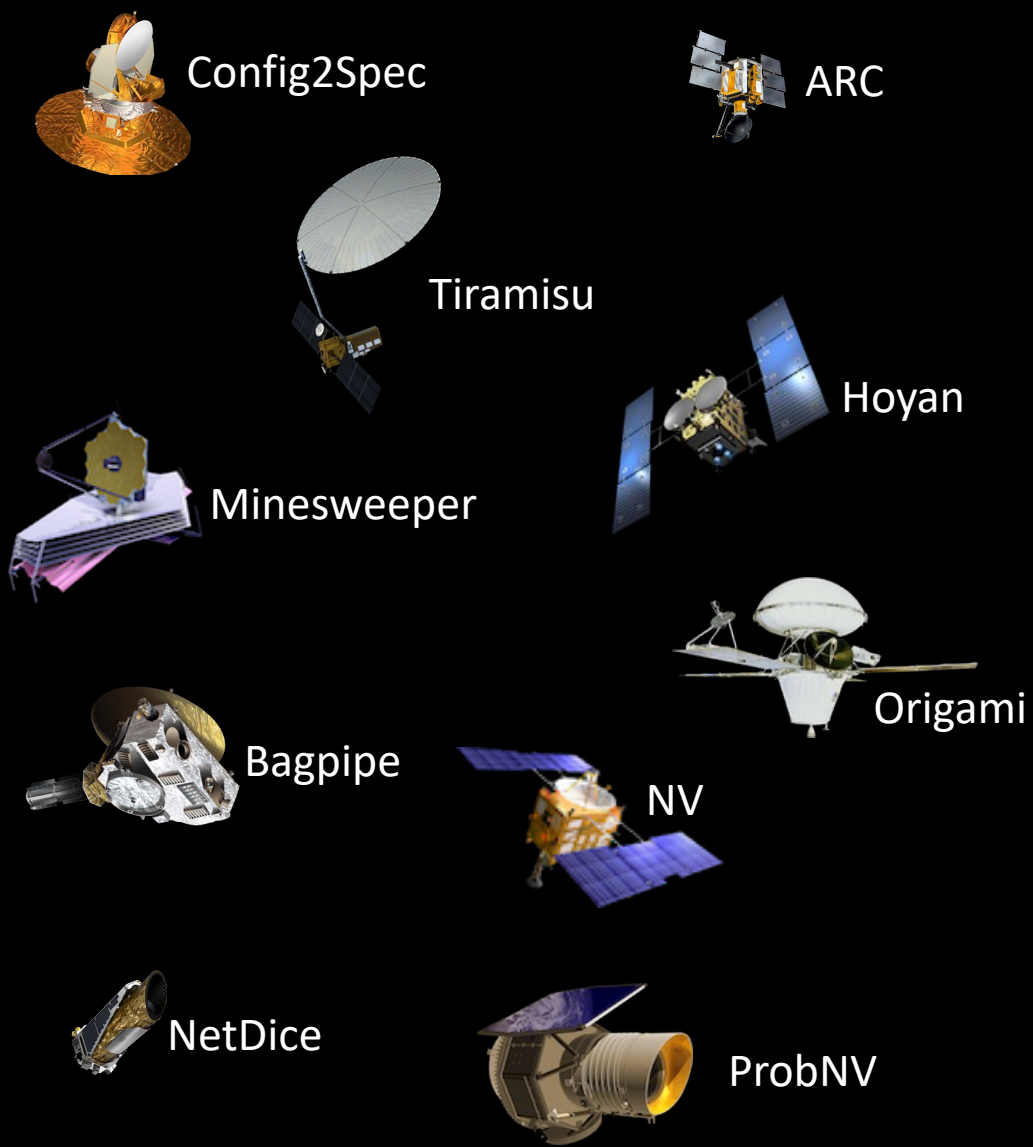


128.0.0.0/2

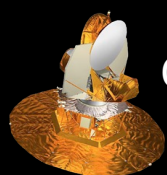


Packet Equivalence Classes





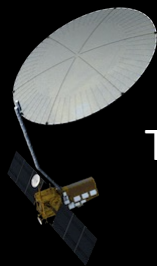
Efficient header
space exploration
Inefficient failure
space exploration



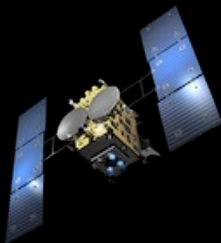
Config2Spec



ARC



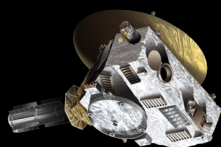
Tiramisu



Hoyan



Minesweeper



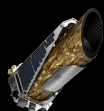
Bagpipe



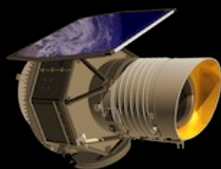
NV



Origami



NetDice

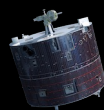


ProbNV

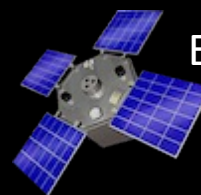
Divide failures into FECs



Bagpipe



Plankton



ERA



ShapeShifter



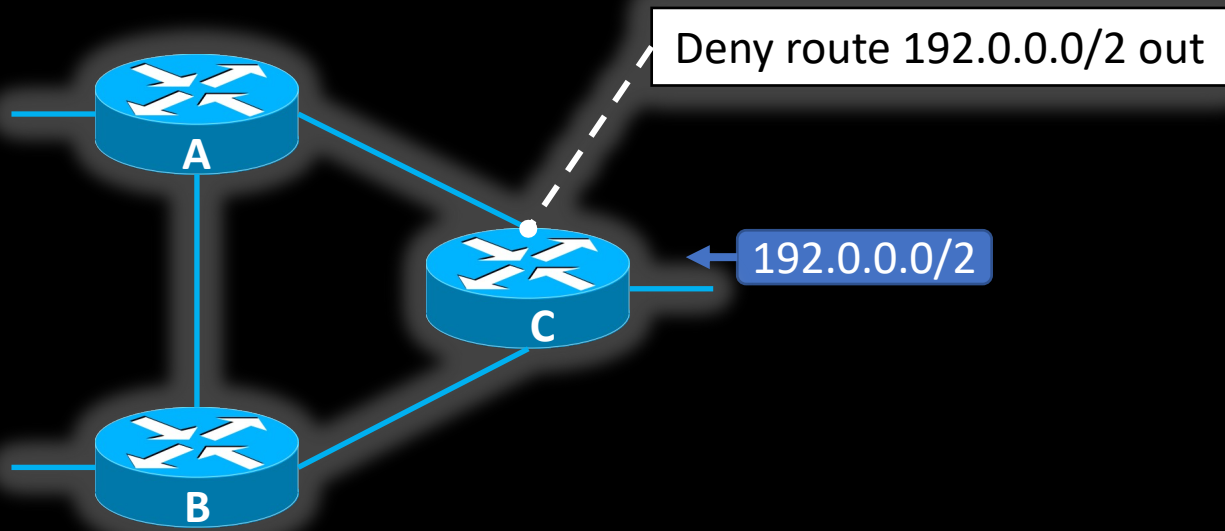
DNA

Divide headers into PECs

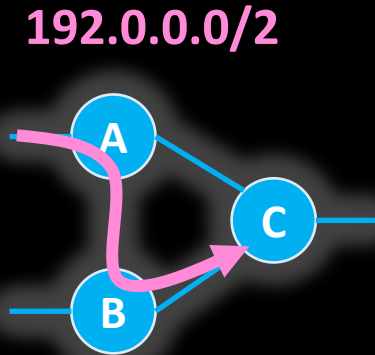
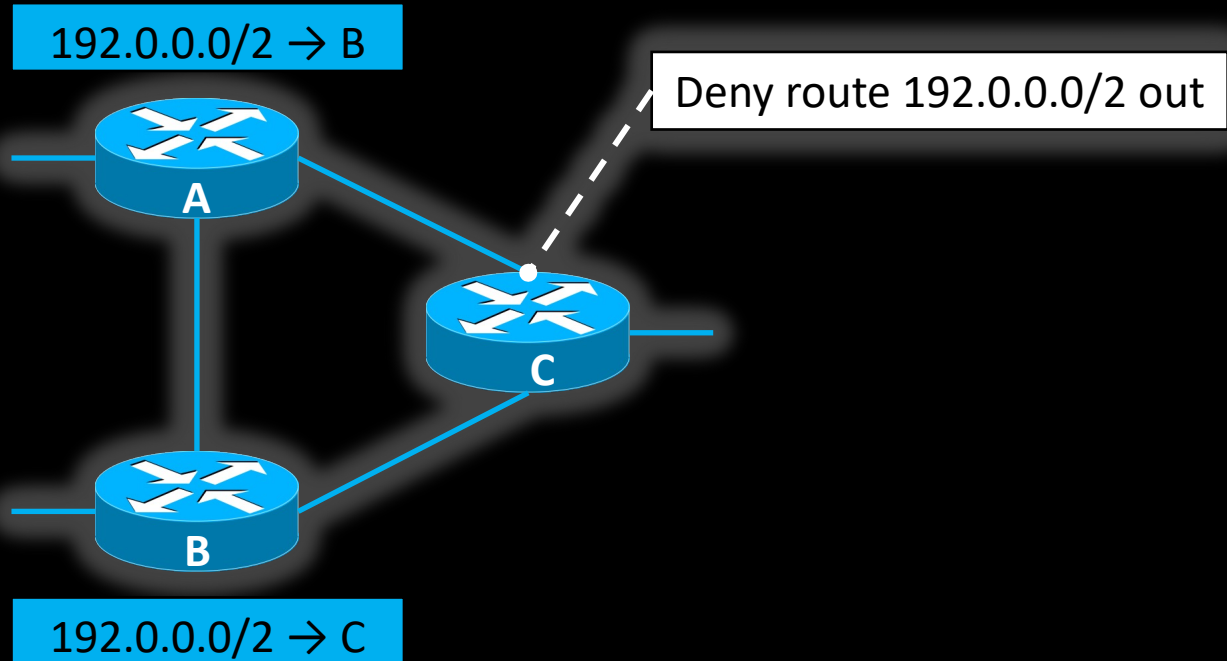
Efficient failure
space exploration

Efficient header
space exploration
Inefficient failure
space exploration

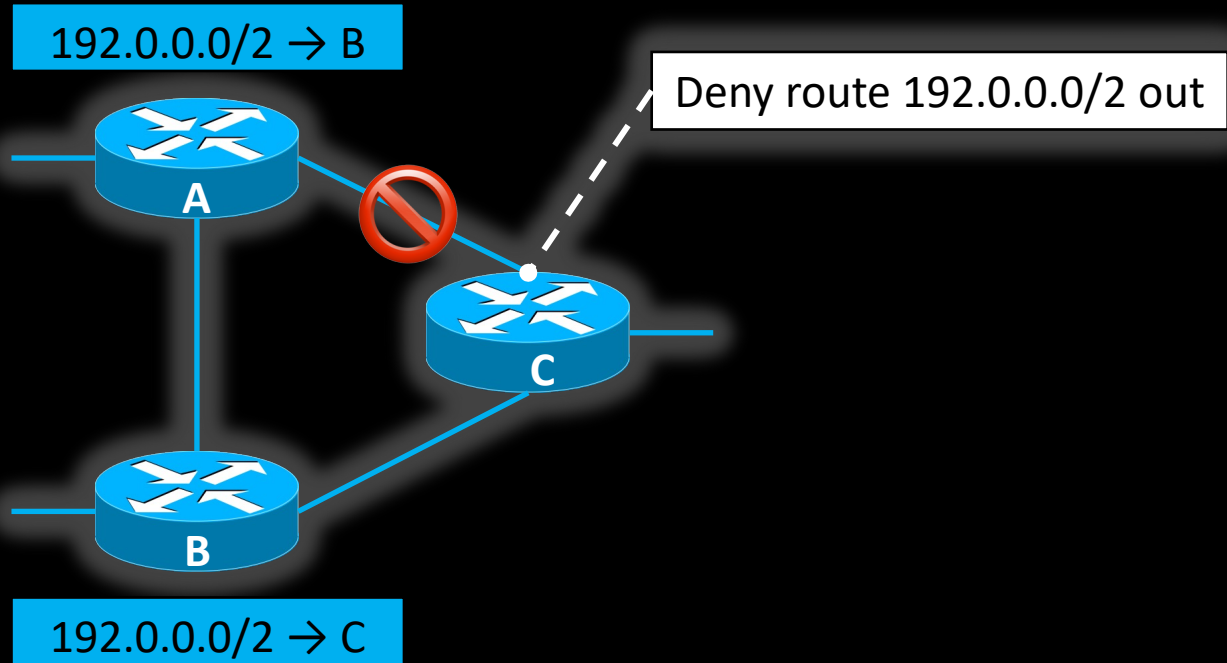
Failure Equivalence Classes



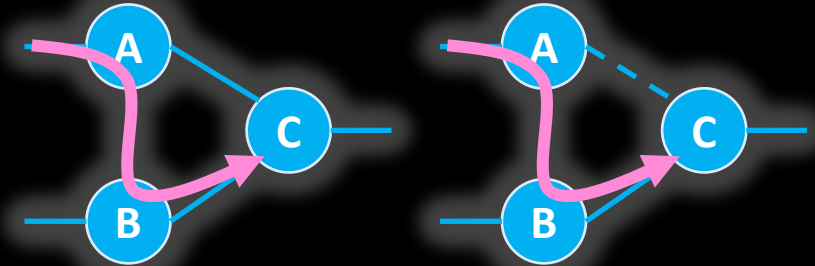
Failure Equivalence Classes



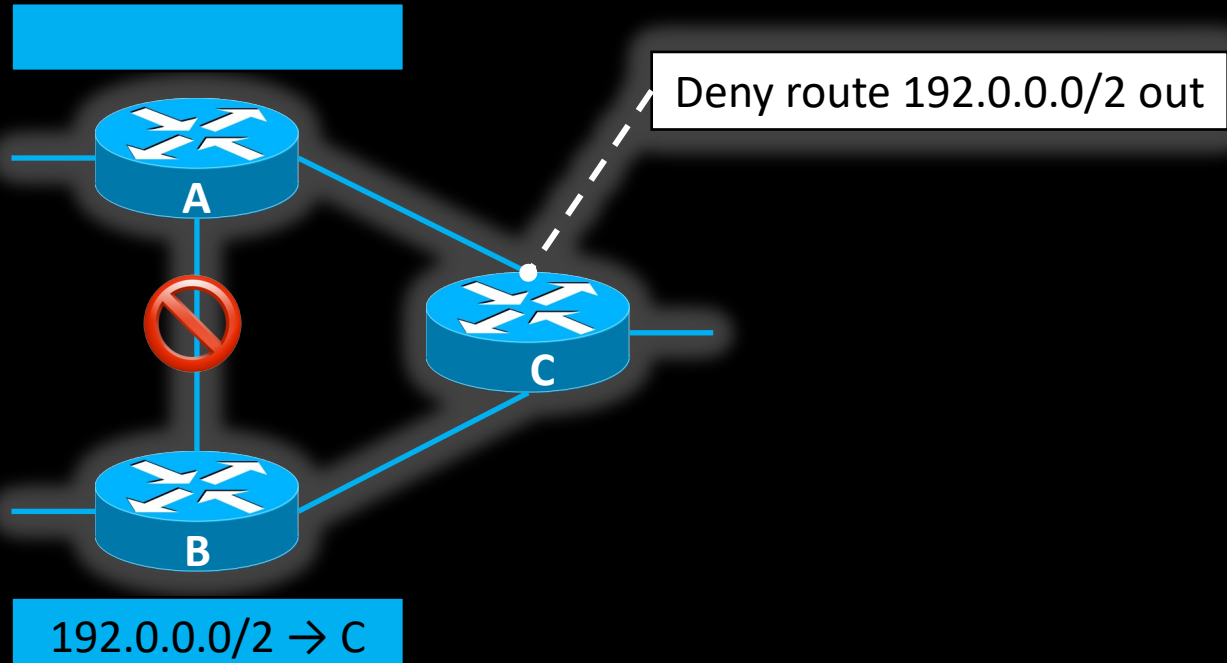
Failure Equivalence Classes



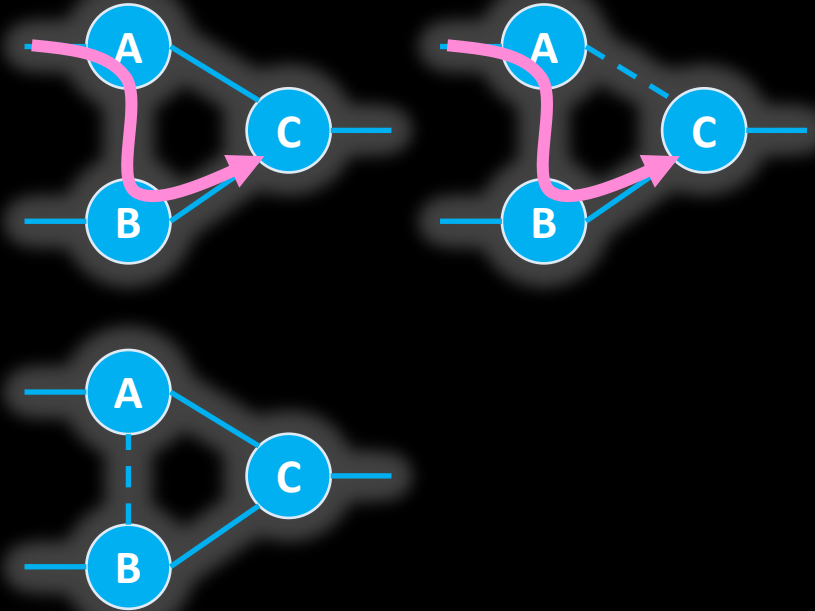
192.0.0.0/2



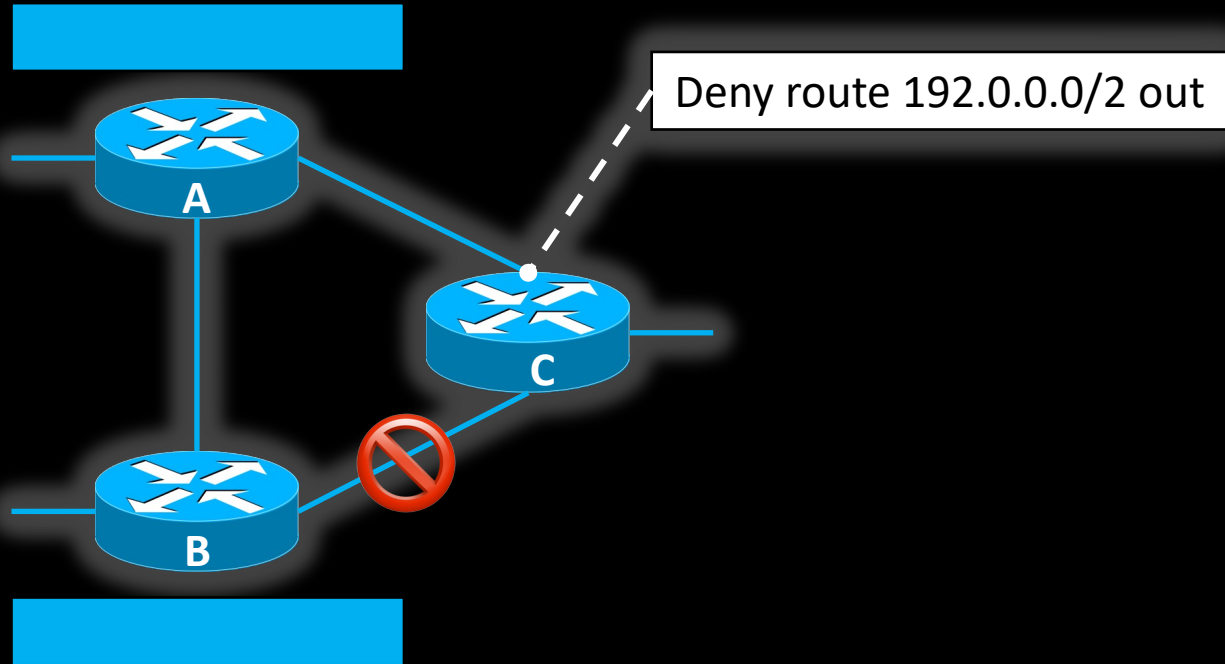
Failure Equivalence Classes



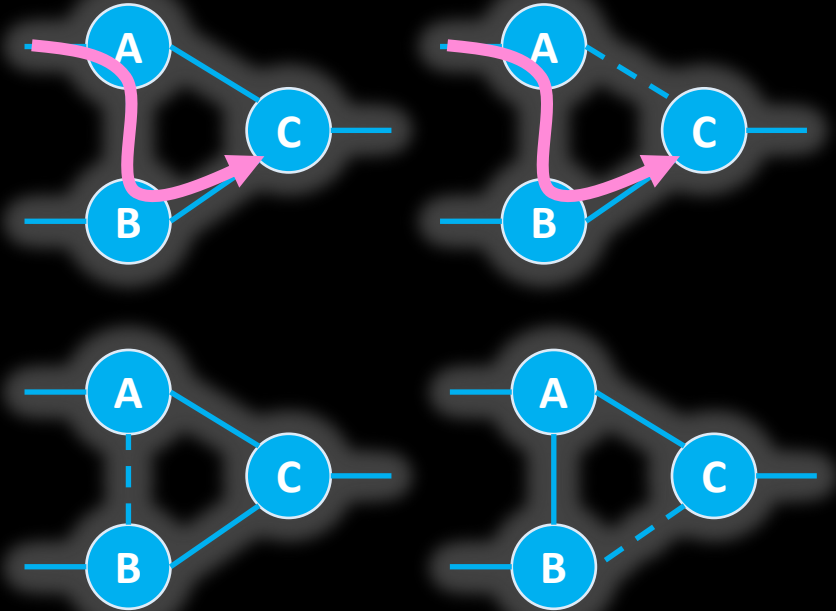
192.0.0.0/2



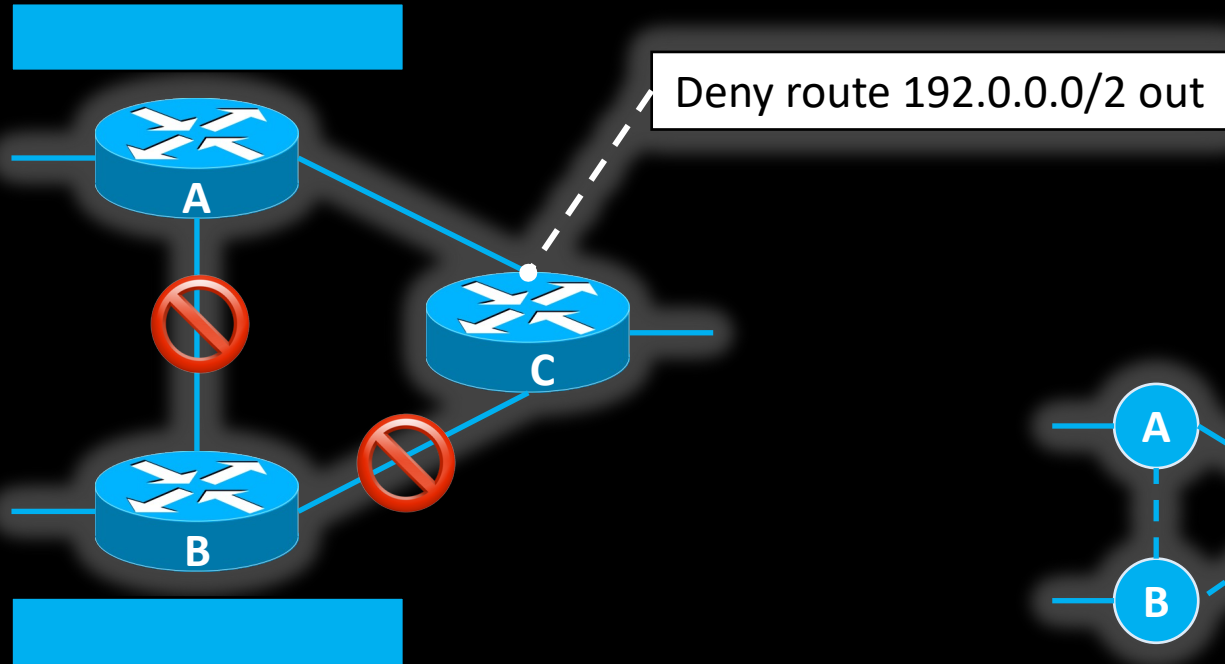
Failure Equivalence Classes



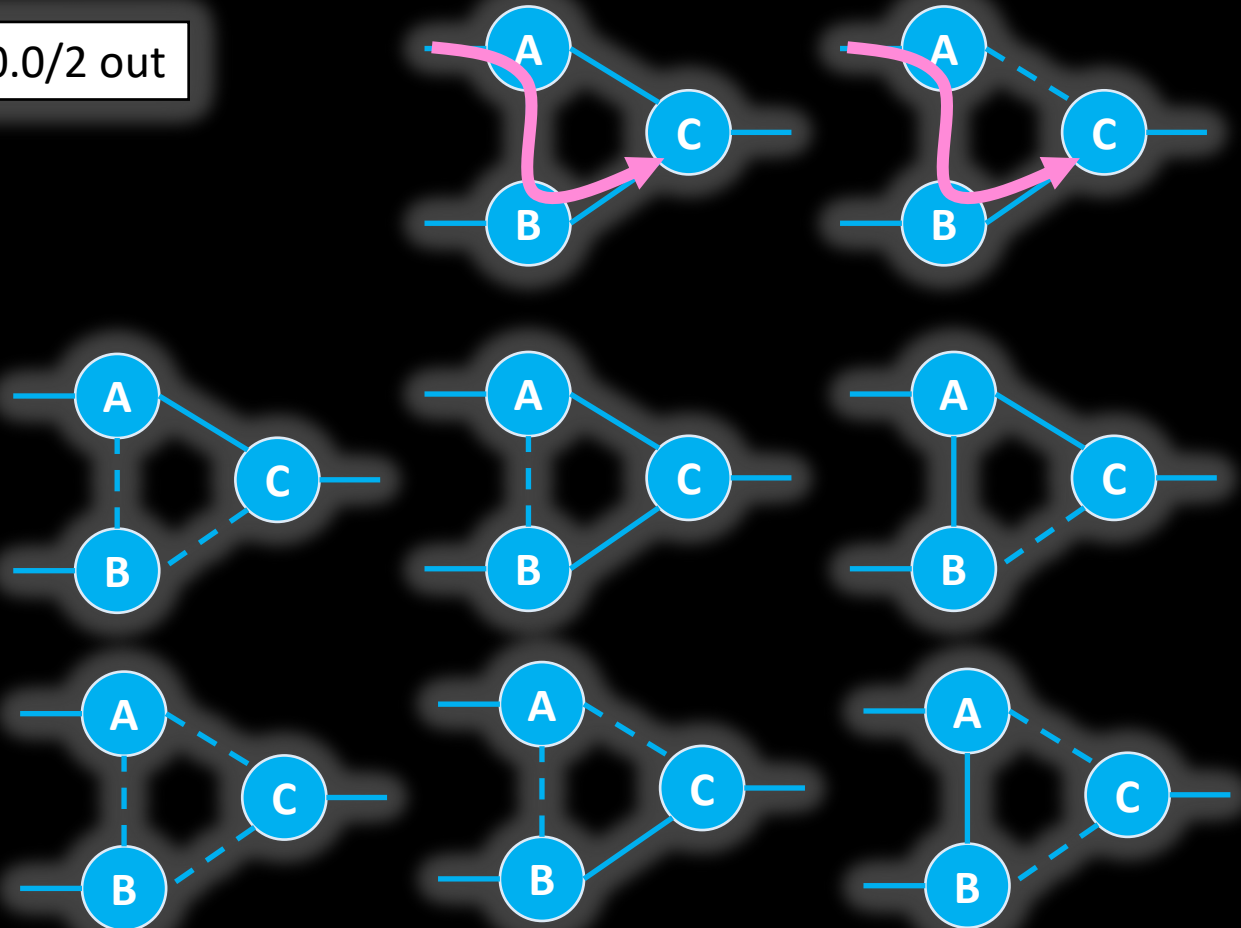
192.0.0.0/2



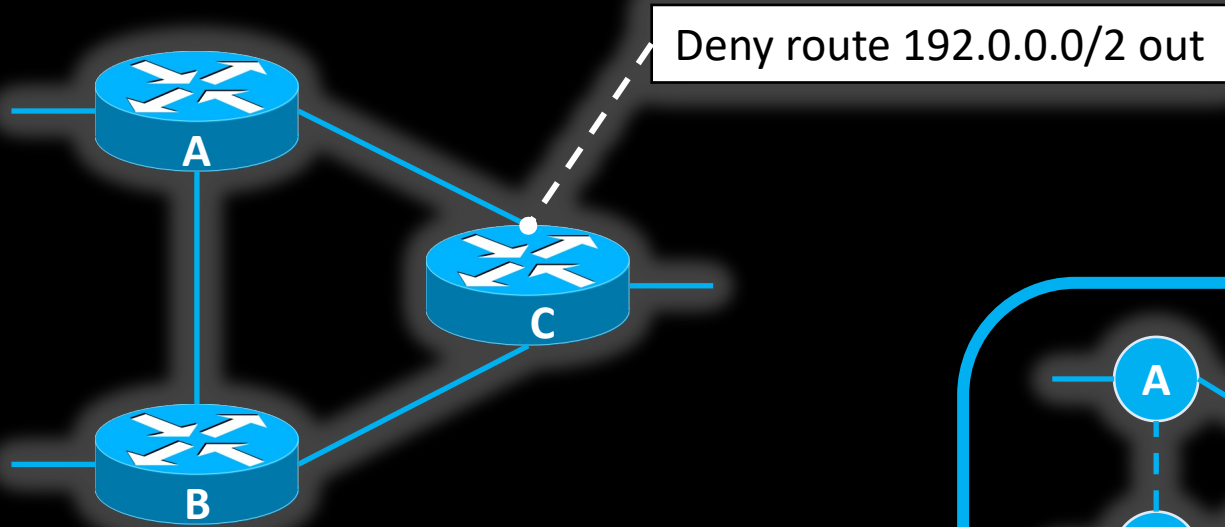
Failure Equivalence Classes



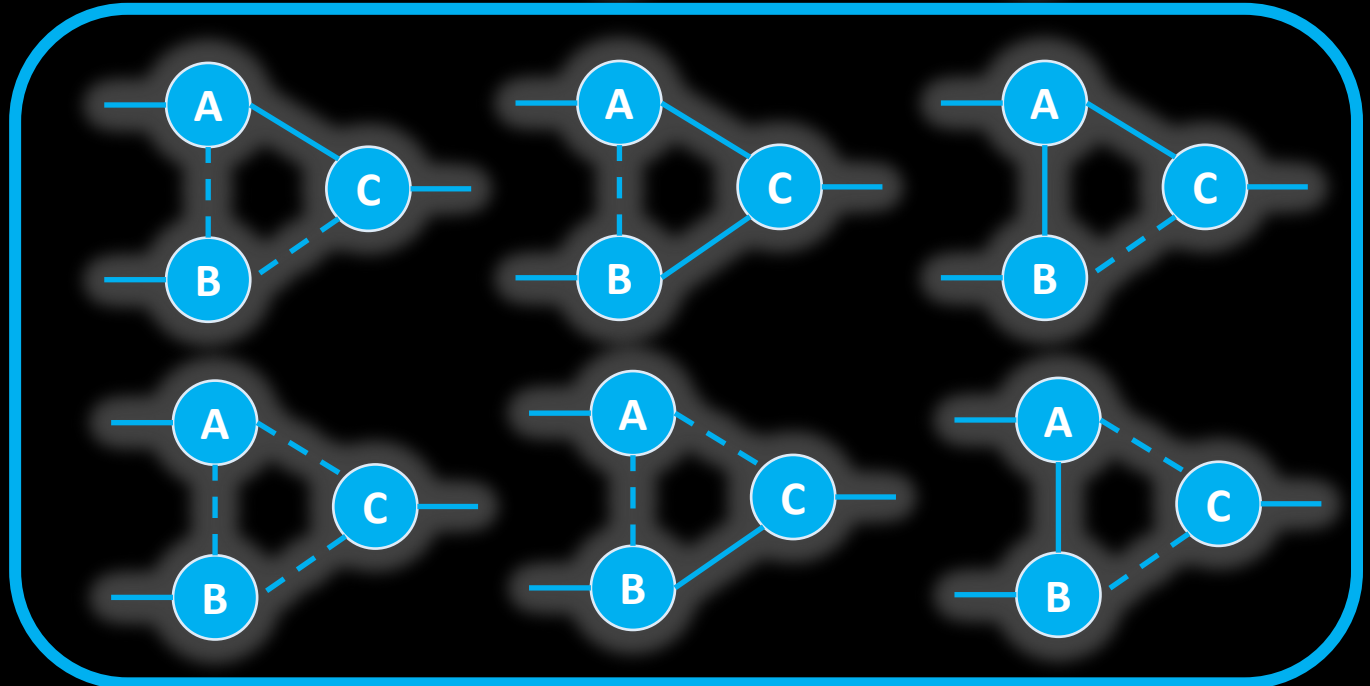
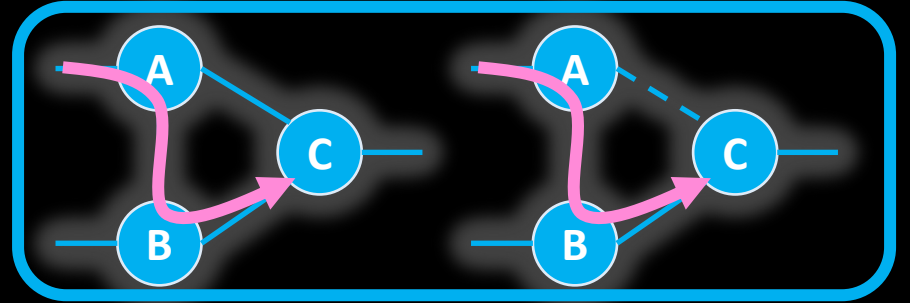
192.0.0.0/2



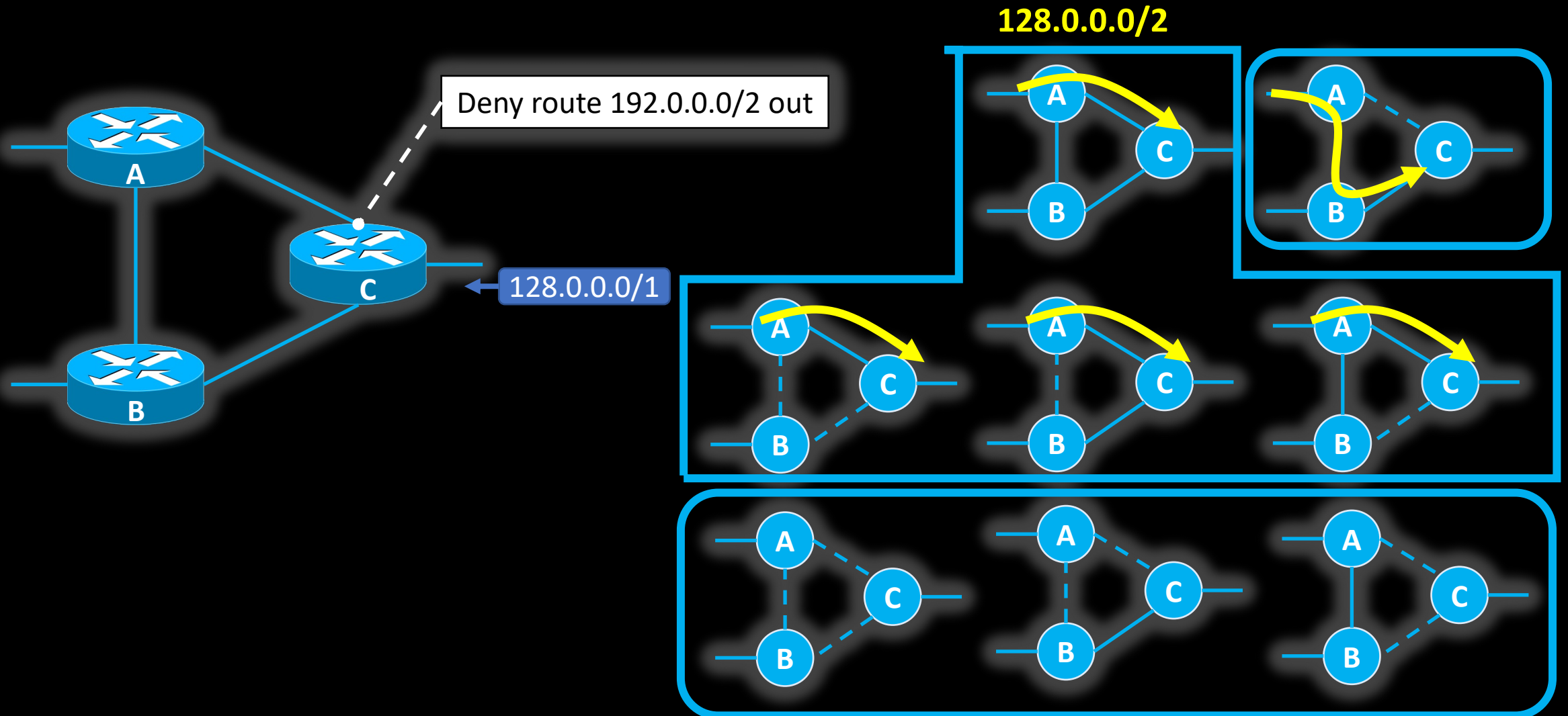
Failure Equivalence Classes

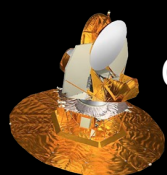


192.0.0.0/2



Failure Equivalence Classes

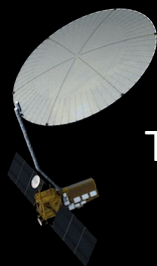




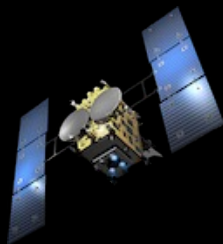
Config2Spec



ARC



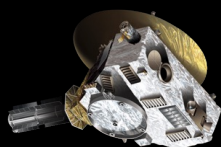
Tiramisu



Hoyan



Minesweeper



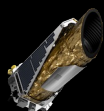
Bagpipe



NV



Origami



NetDice

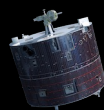


ProbNV

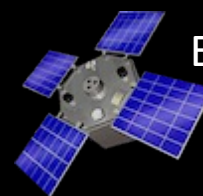
Divide failures into FECs



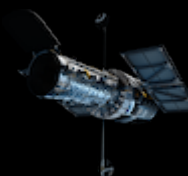
Bagpipe



Plankton



ERA



ShapeShifter

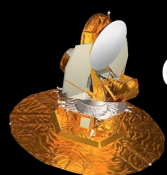


DNA

Divide headers into PECs

← Inefficient header
space exploration

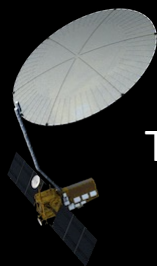
← Efficient failure
space exploration



Config2Spec



ARC



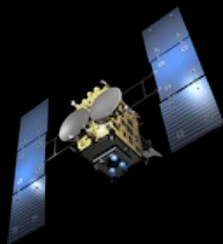
Tiramisu



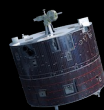
Bagpipe



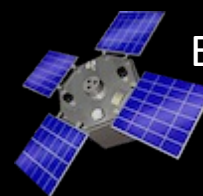
Minesweeper



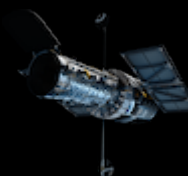
Hoyan



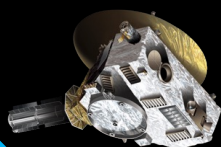
Plankton



ERA



ShapeShifter



Bagpipe



Origami

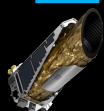


DNA



NV

Deterministic failures



NetDice



ProbNV

Probabilistic failures



Scales to the product space
of headers and failures
by computing PFECs



SRE

Scales to the product space
of headers and failures
by computing PFECs



SRE

Generalizes to deterministic and
probabilistic failures by delaying
binding to a failure model

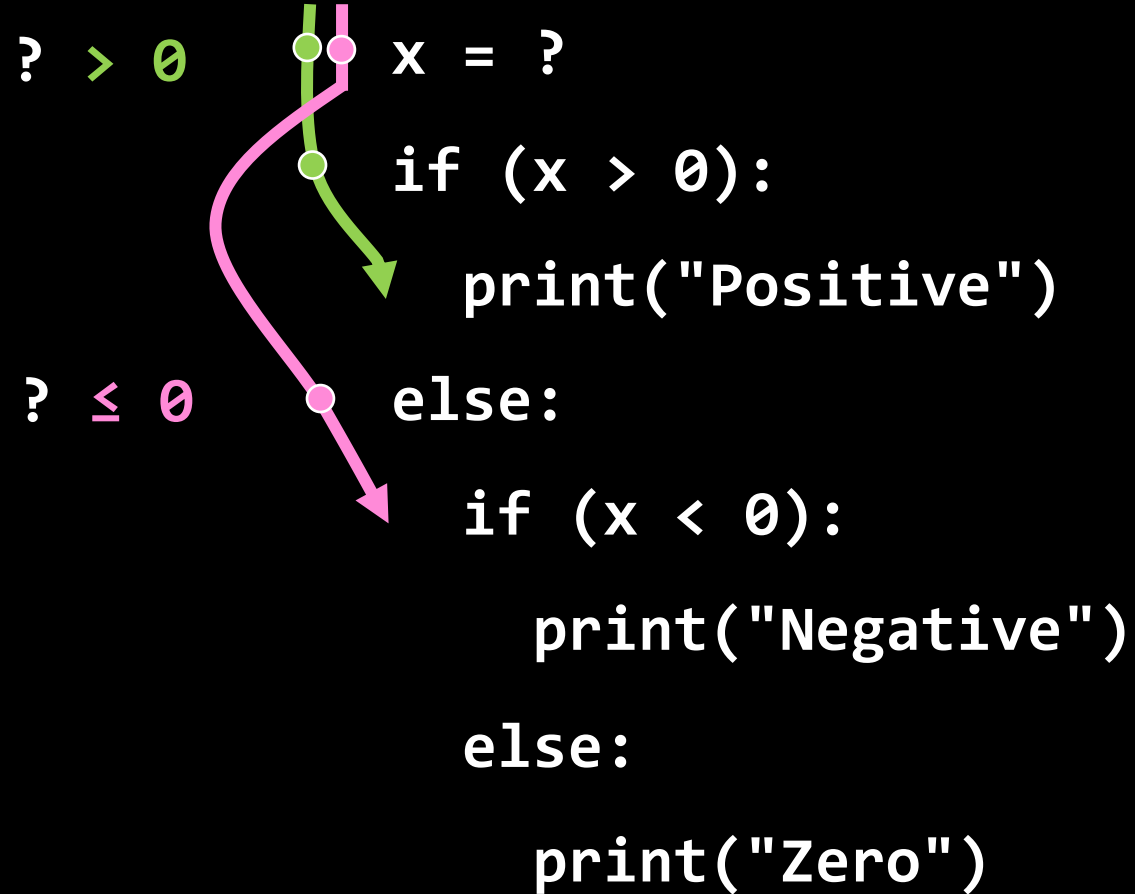
Basic idea: Symbolic execution

```
x = input("Integer:")  
if (x > 0):  
    print("Positive")  
else:  
    if (x < 0):  
        print("Negative")  
    else:  
        print("Zero")
```

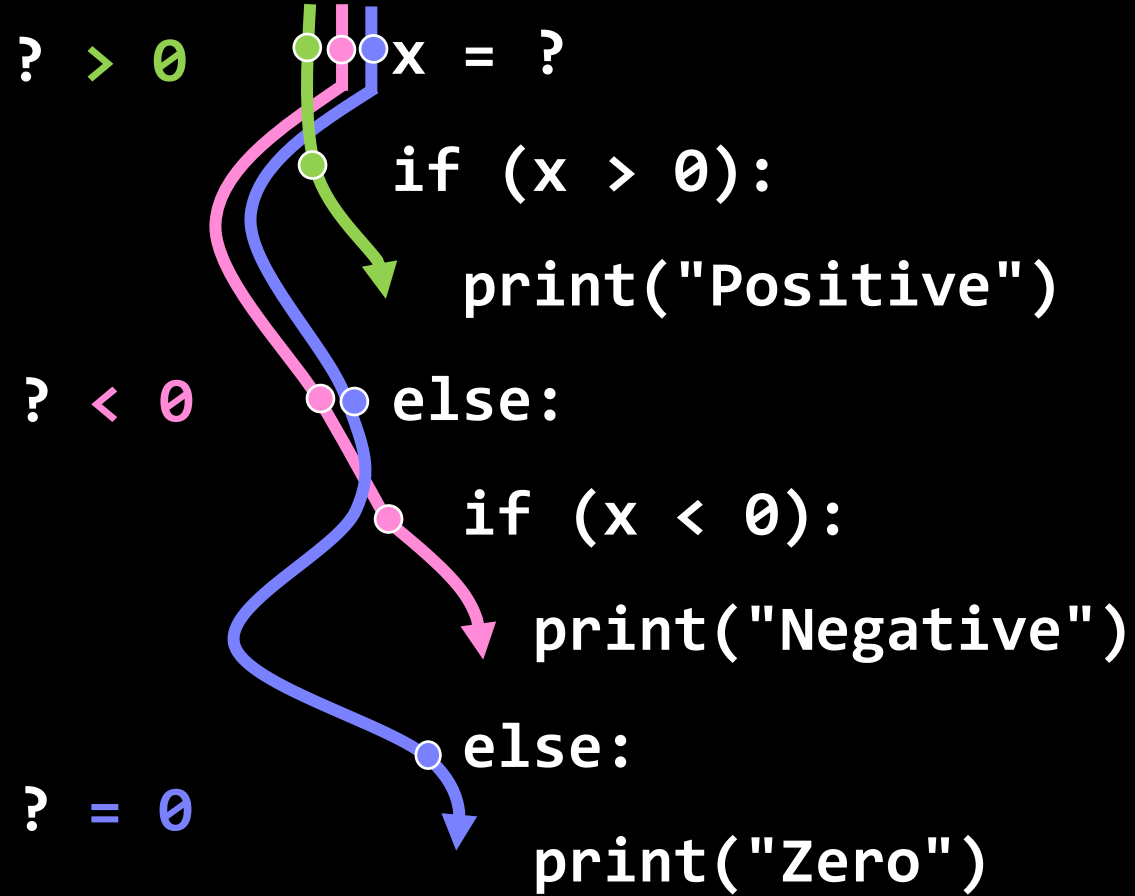
Basic idea: Symbolic execution

```
x = ?  
  
if (x > 0):  
    print("Positive")  
else:  
    if (x < 0):  
        print("Negative")  
    else:  
        print("Zero")
```

Basic idea: Symbolic execution



Basic idea: Symbolic execution

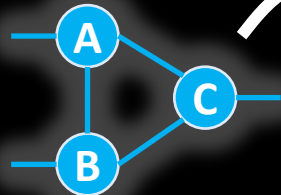


Symbolic execution of control and data planes



Symbolic execution of control and data planes

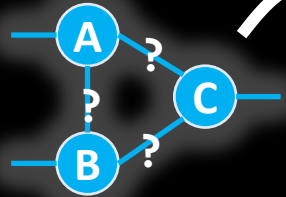
Configurations



Available links

Symbolic execution of control and data planes

Configurations



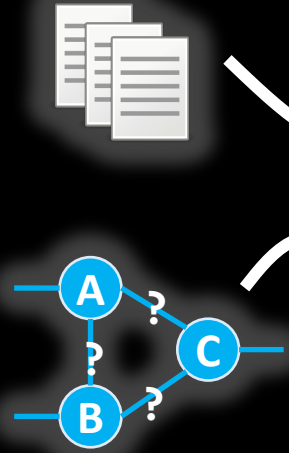
Available links

Symbolic
routing

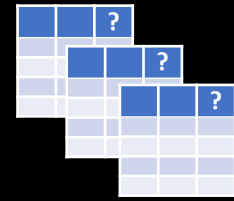
Data
Plane

Symbolic execution of control and data planes

Configurations



Symbolic routing



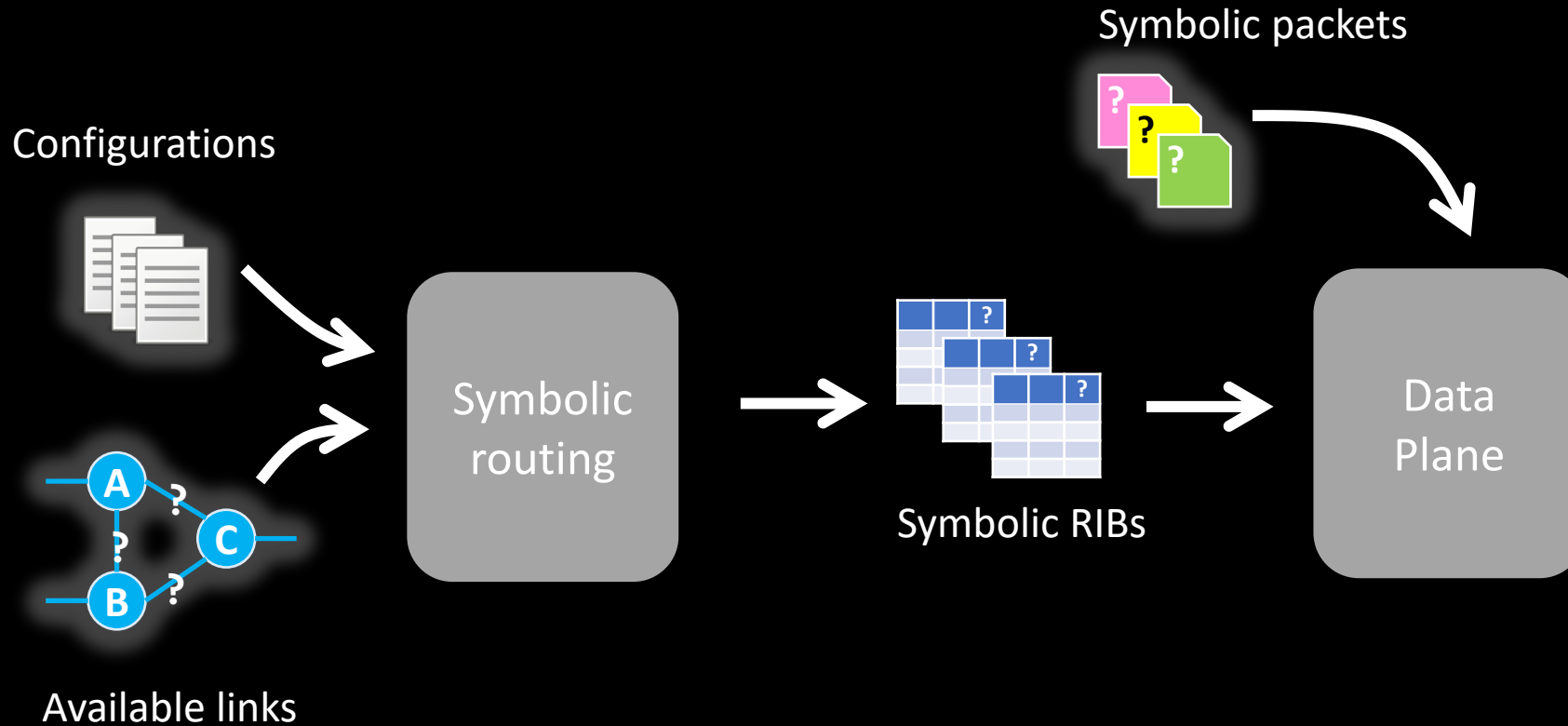
Symbolic RIBs

Data Plane

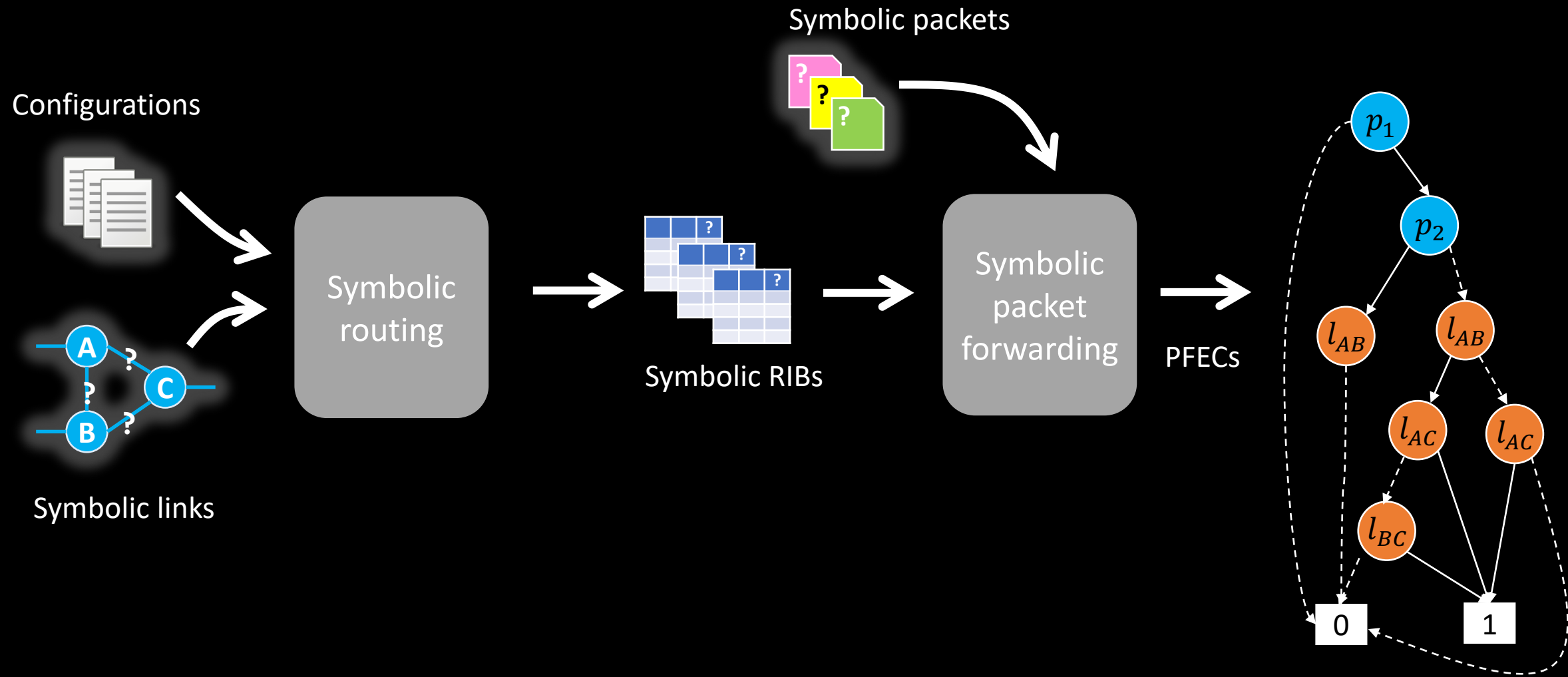


Available links

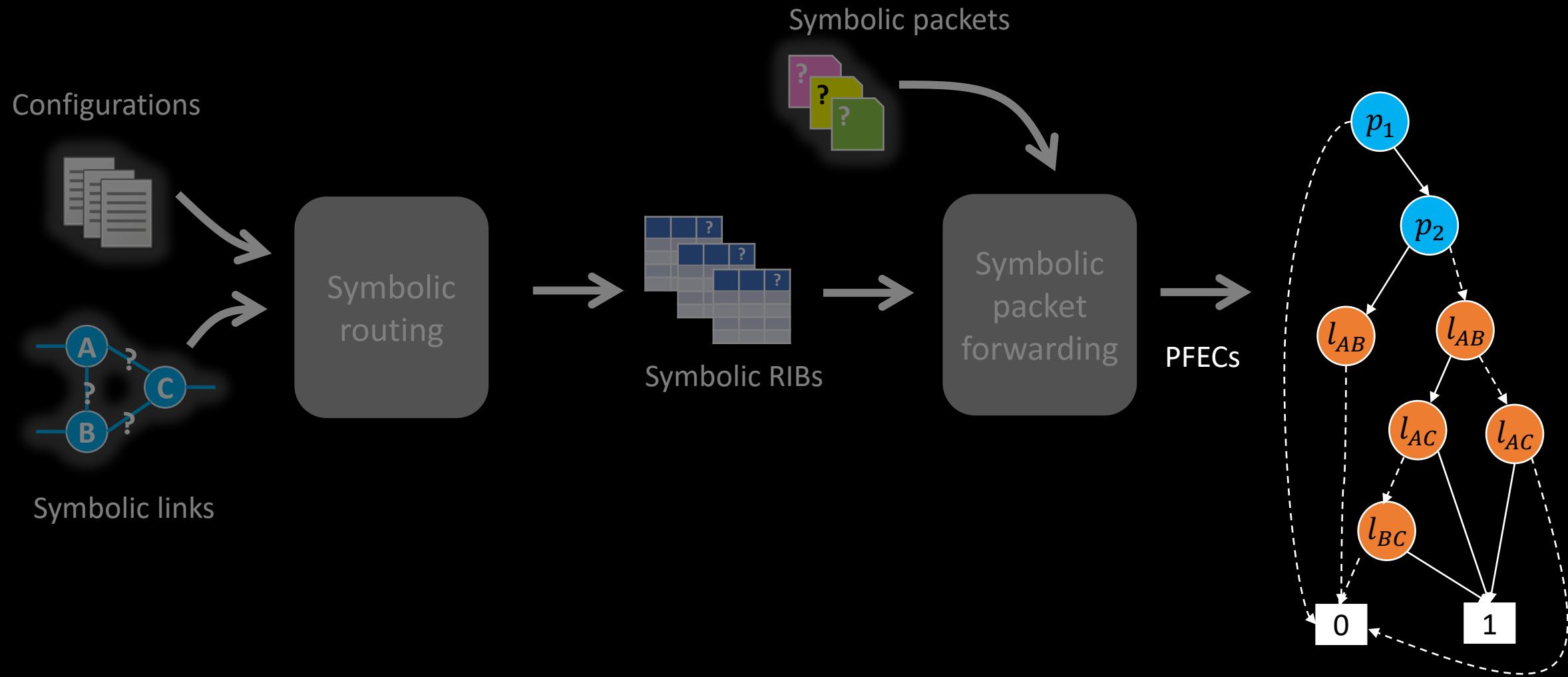
Symbolic execution of control and data planes



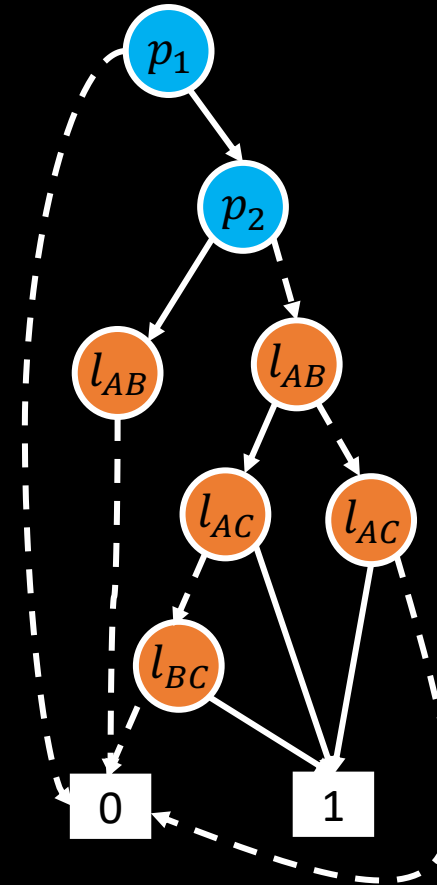
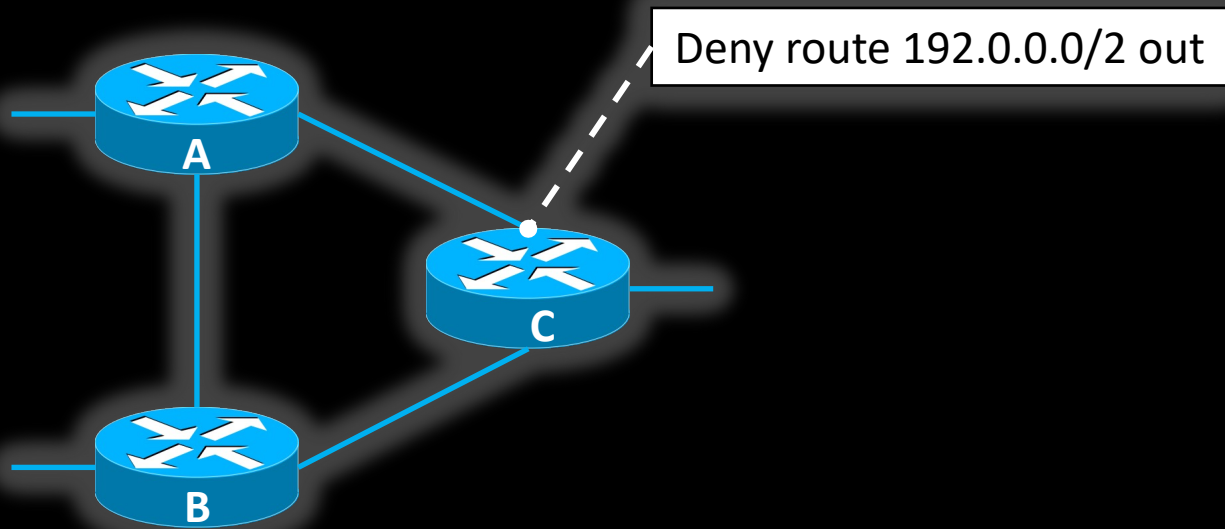
Symbolic execution of control and data planes



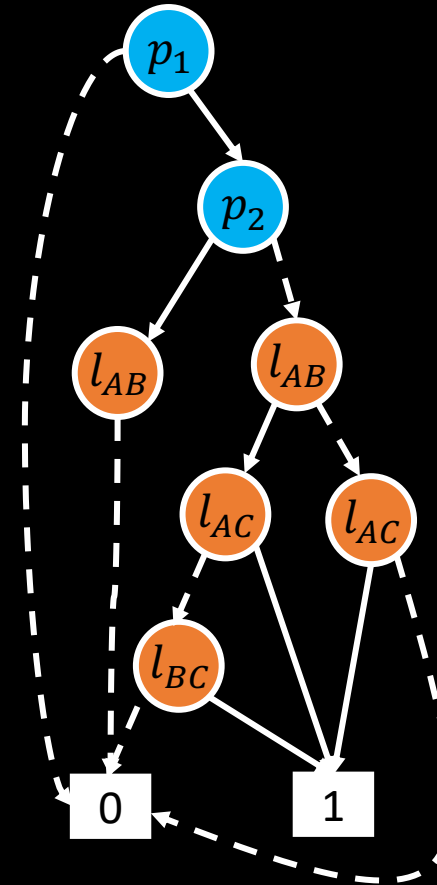
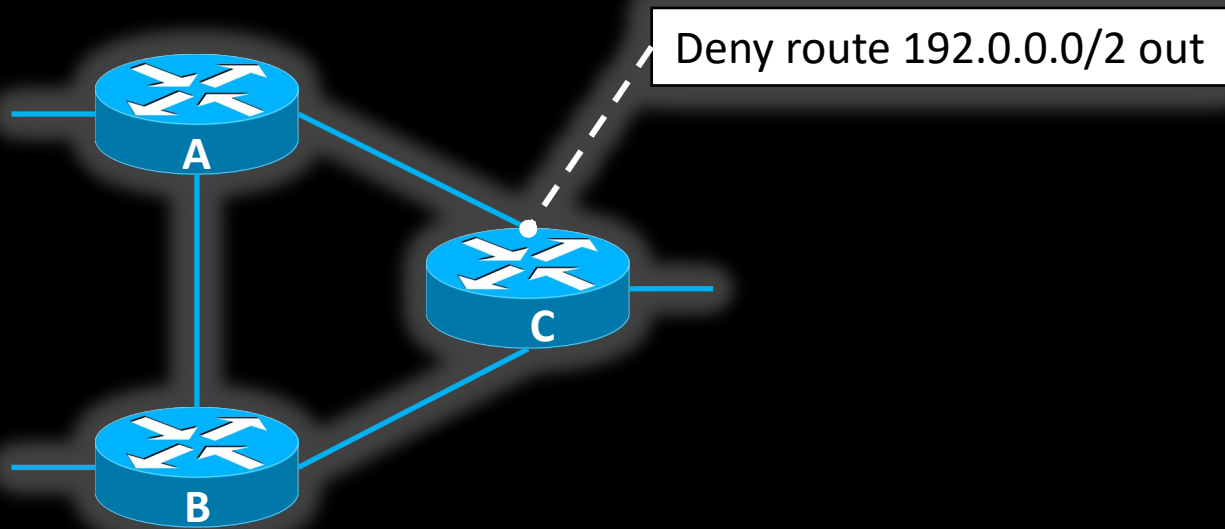
Symbolic execution of control and data planes



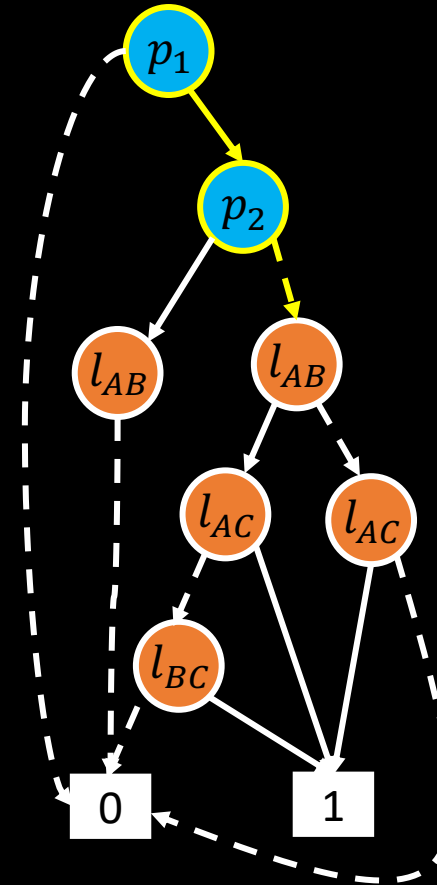
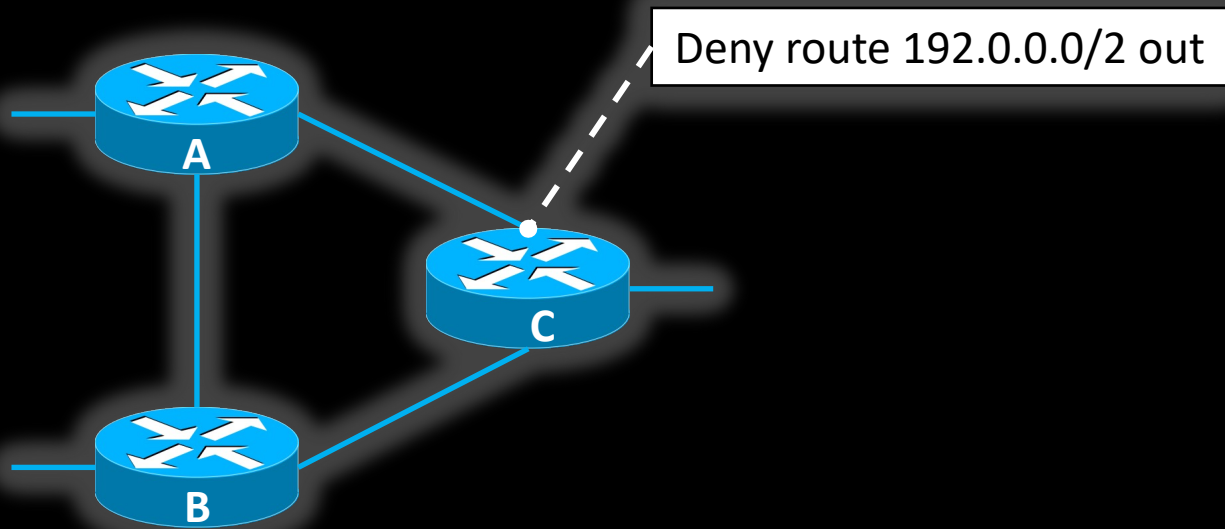
How many failures can occur
and A still reach 128.0.0.0/2?



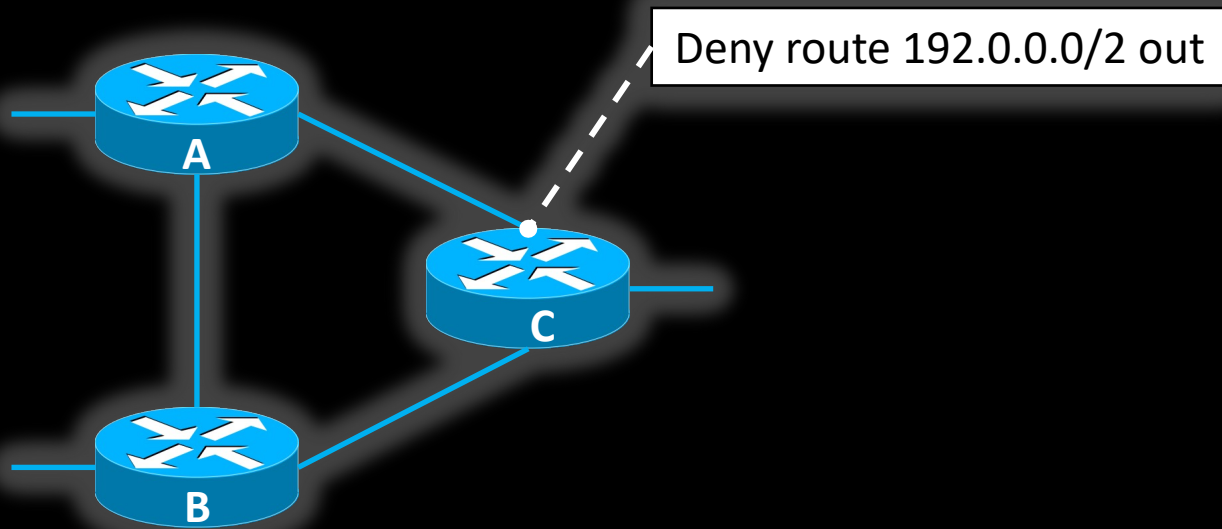
How many failures can occur
and A still reach 128.0.0.0/2?



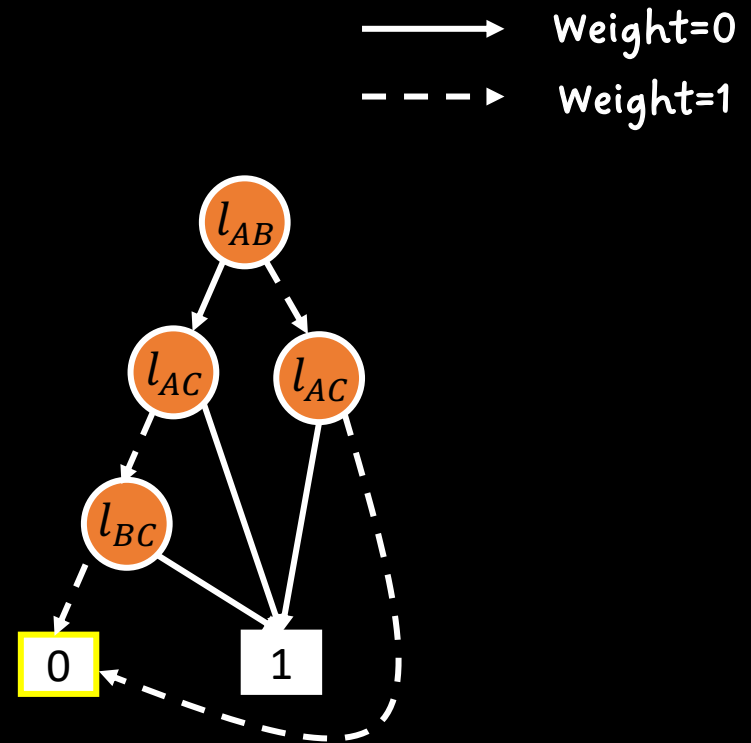
How many failures can occur
and A still reach 128.0.0.0/2?



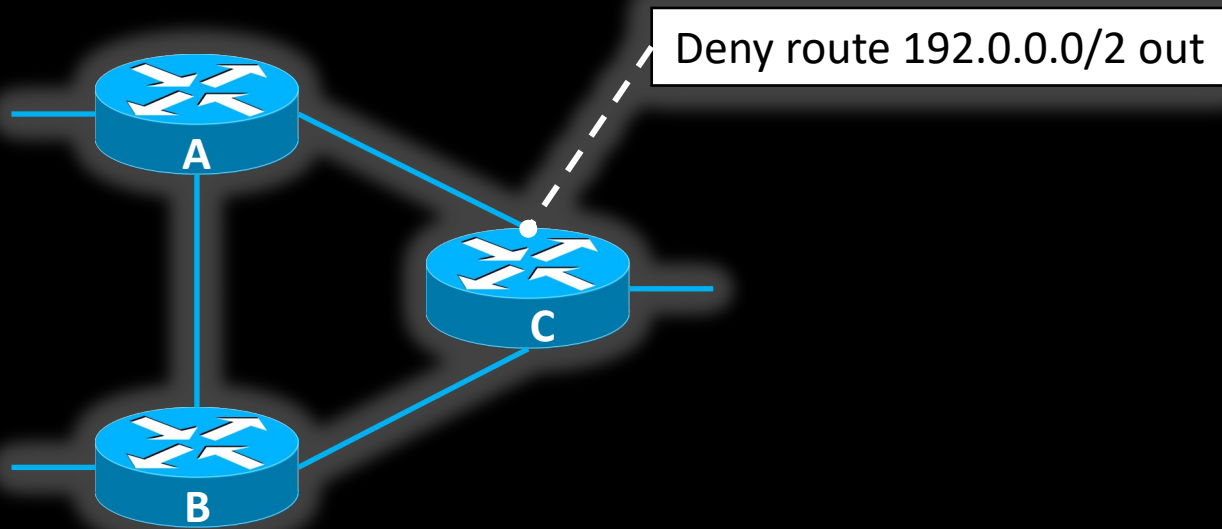
How many failures can occur
and A still reach 128.0.0.0/2?



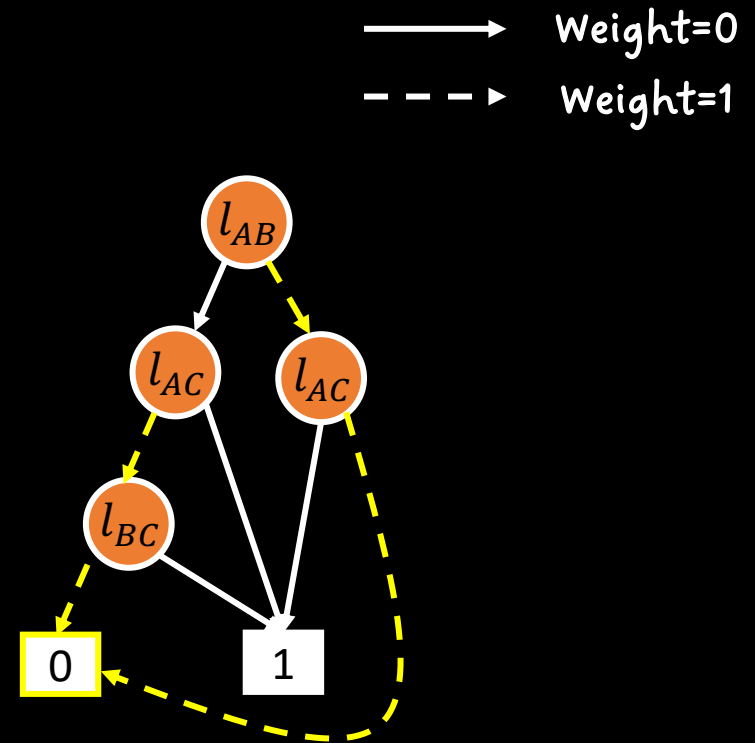
Failure tolerance =
(Len of shortest path to terminal 0) - 1



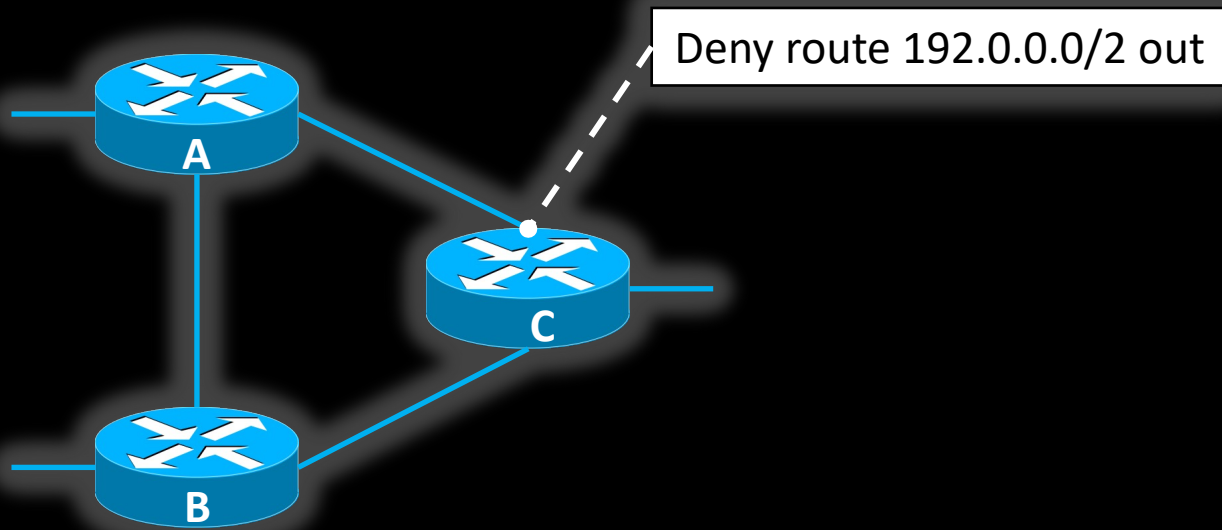
How many failures can occur
and A still reach 128.0.0.0/2?



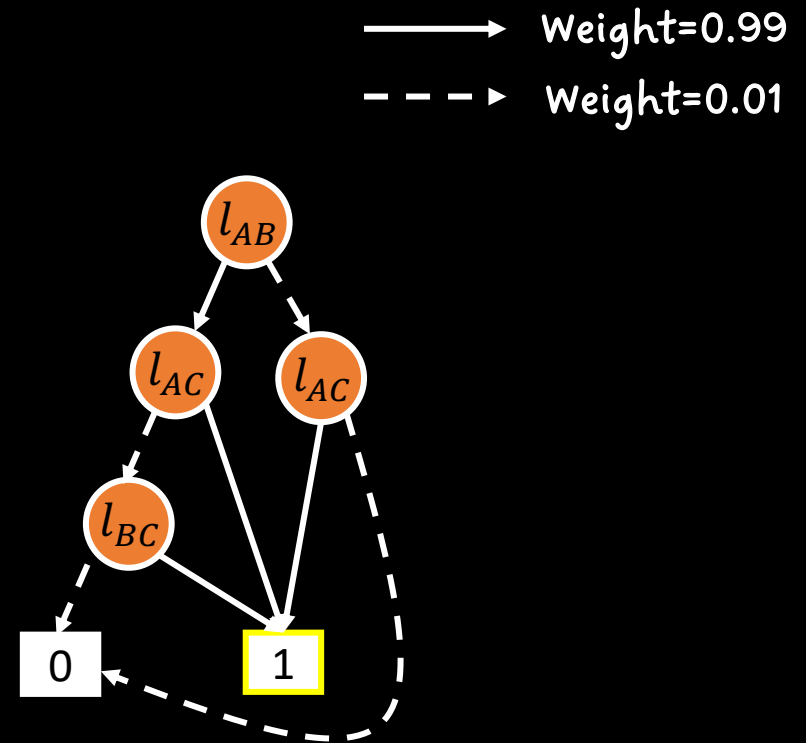
Failure tolerance =
 $2 - 1$



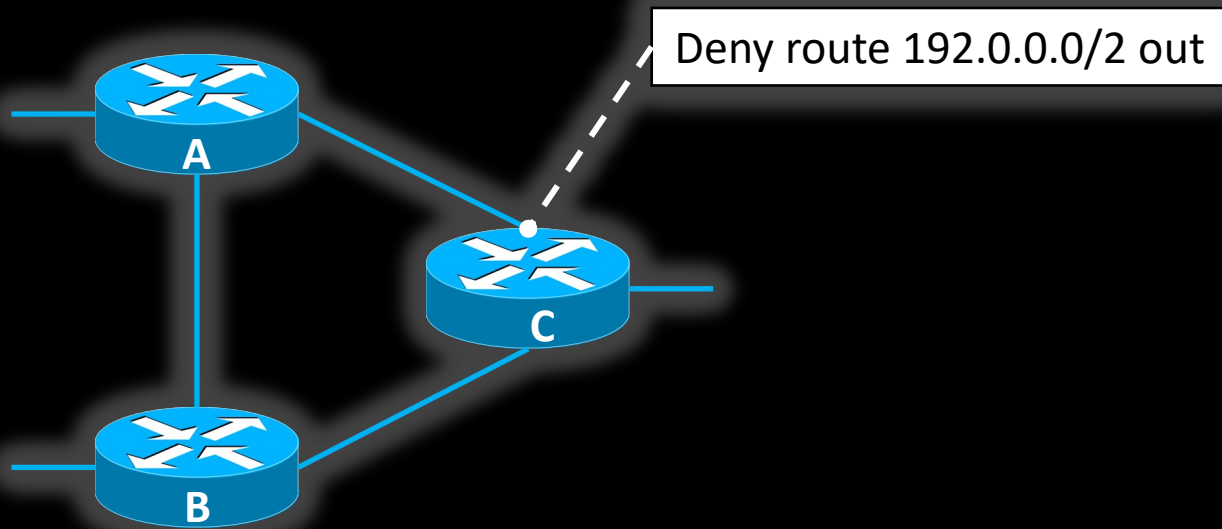
What is the probability
A can reach 128.0.0.0/2?



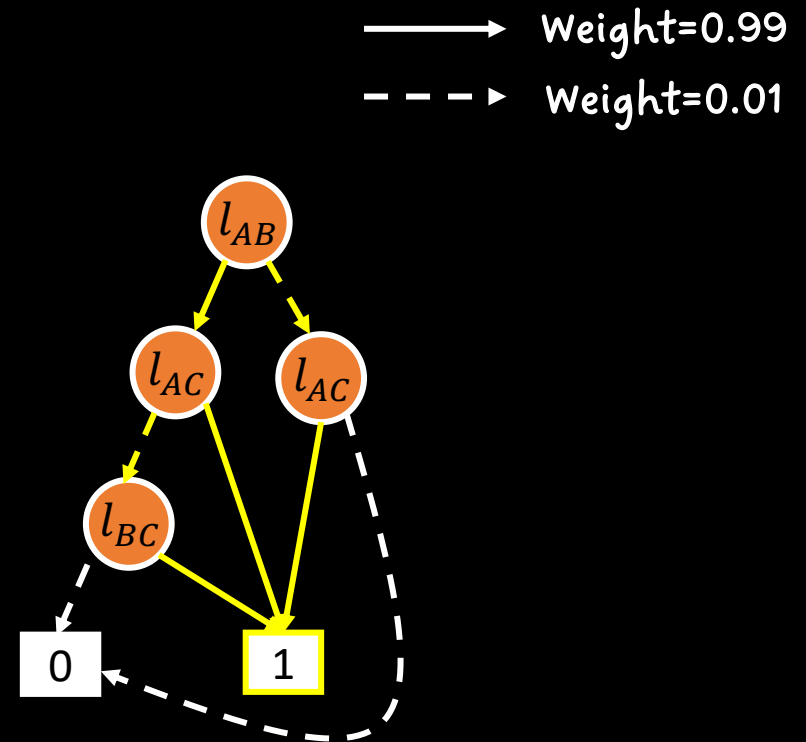
Probability =
Weighted sum of all paths to terminal 1



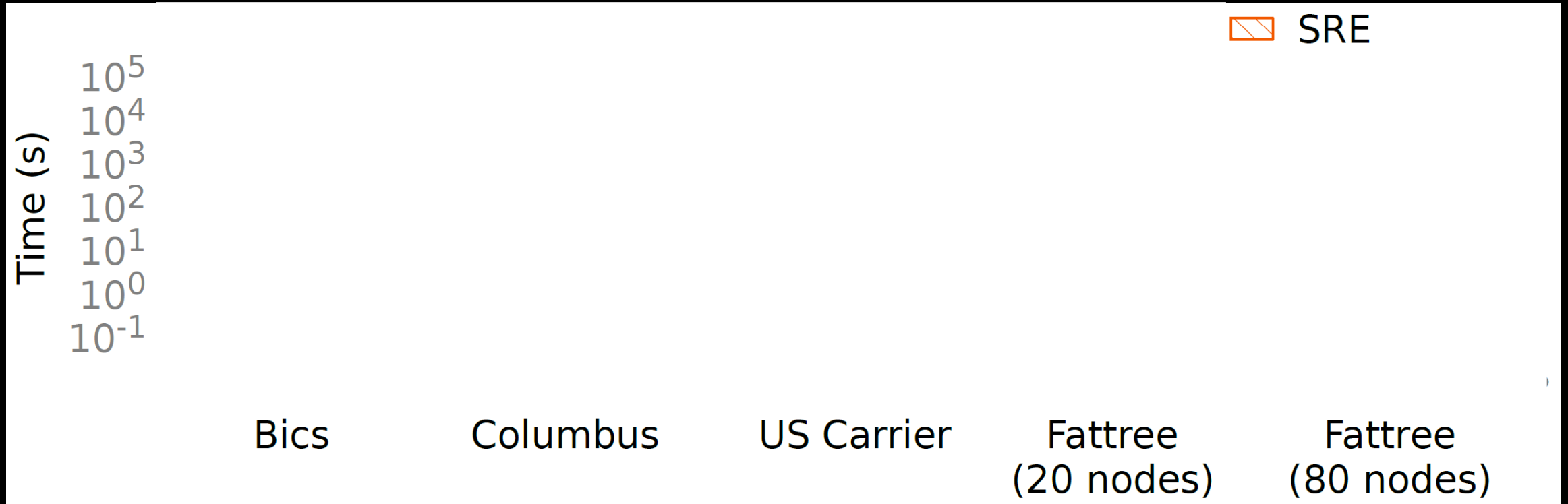
What is the probability
A can reach 128.0.0.0/2?



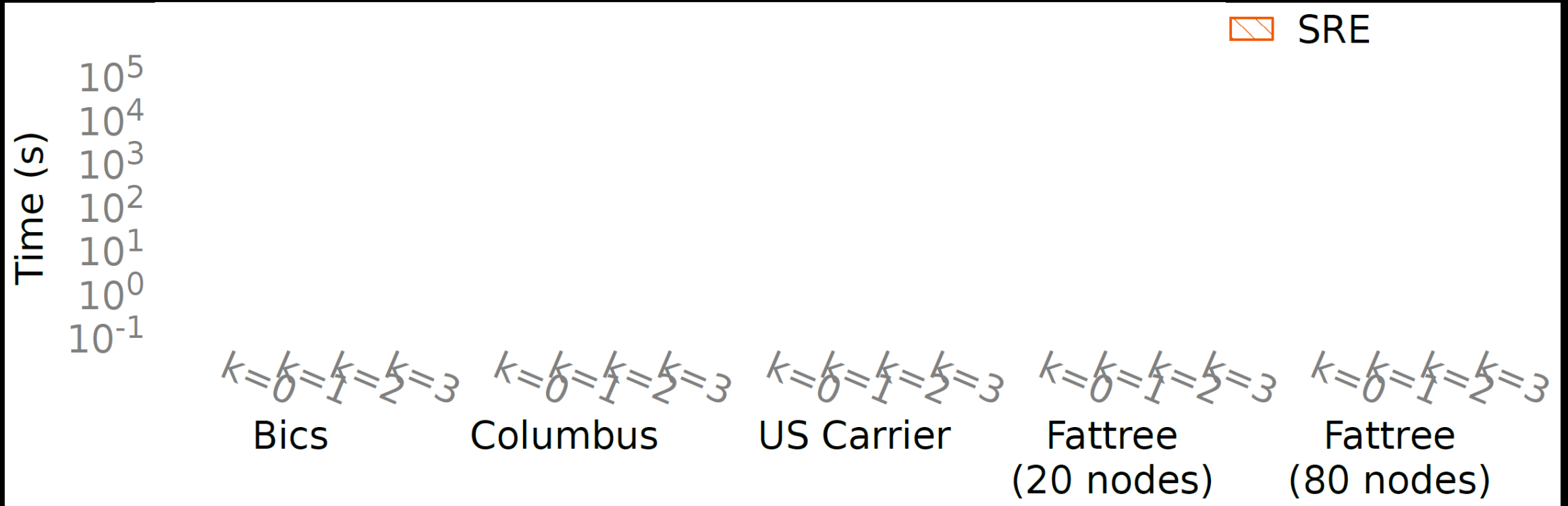
$$\text{Probability} = 0.99 * (0.01 * 0.99 + 0.99) + 0.01 * 0.99$$



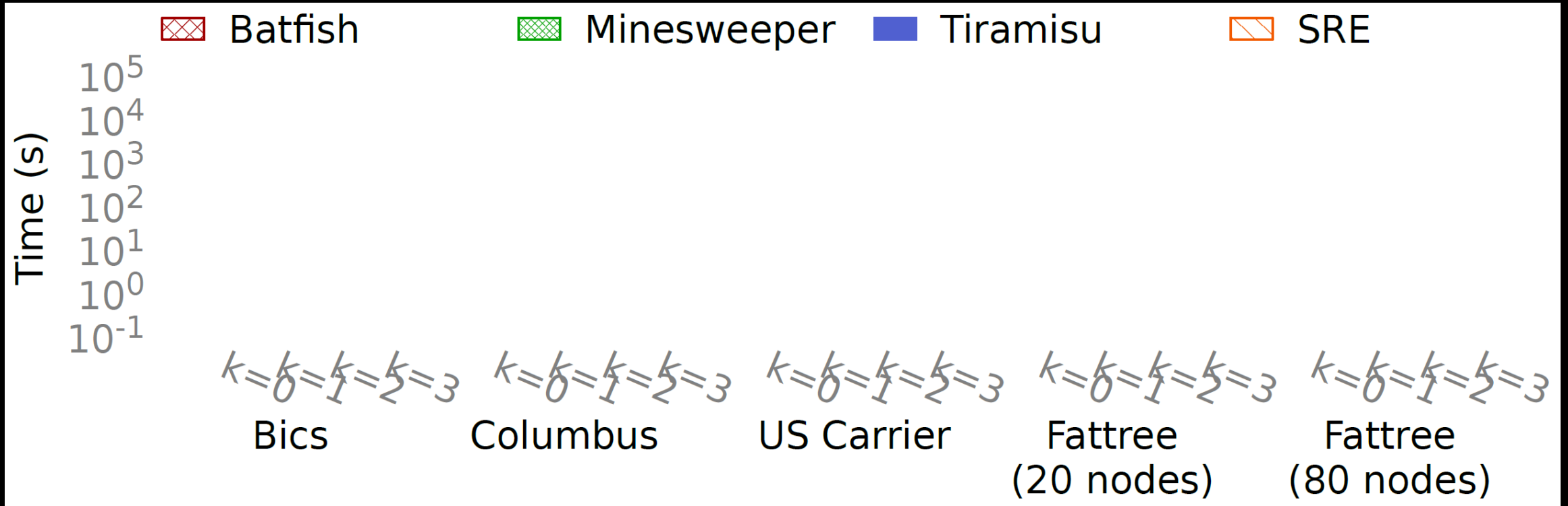
Experiment: compute all-pairs reachability



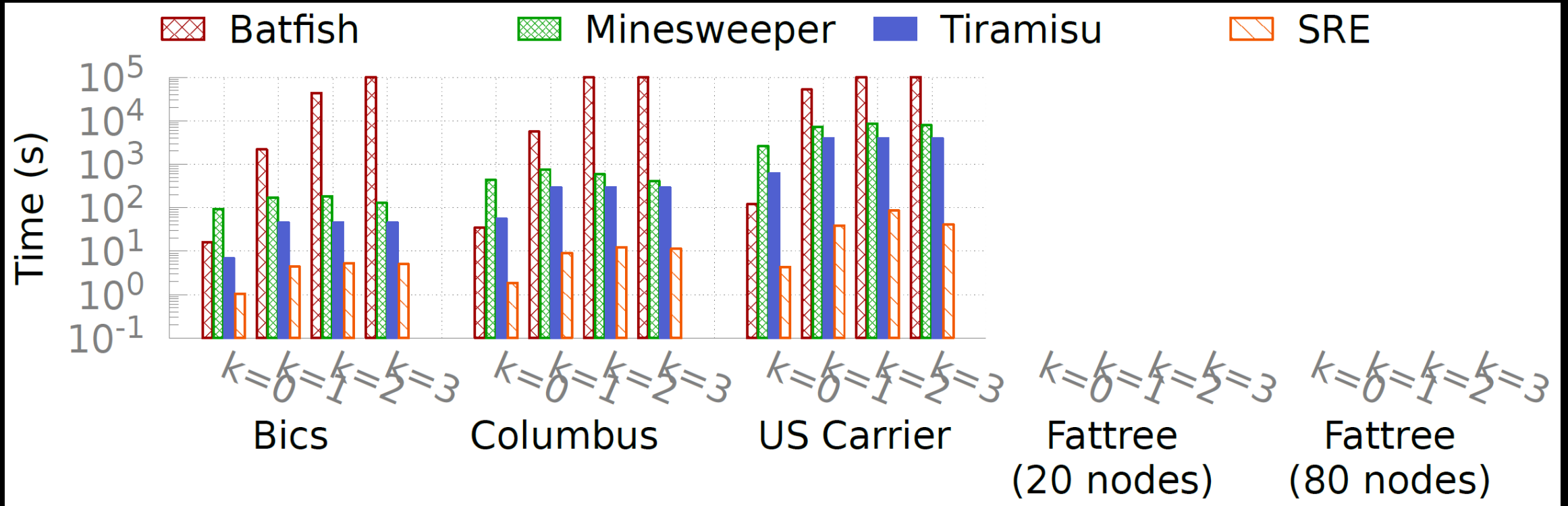
Experiment: compute all-pairs reachability



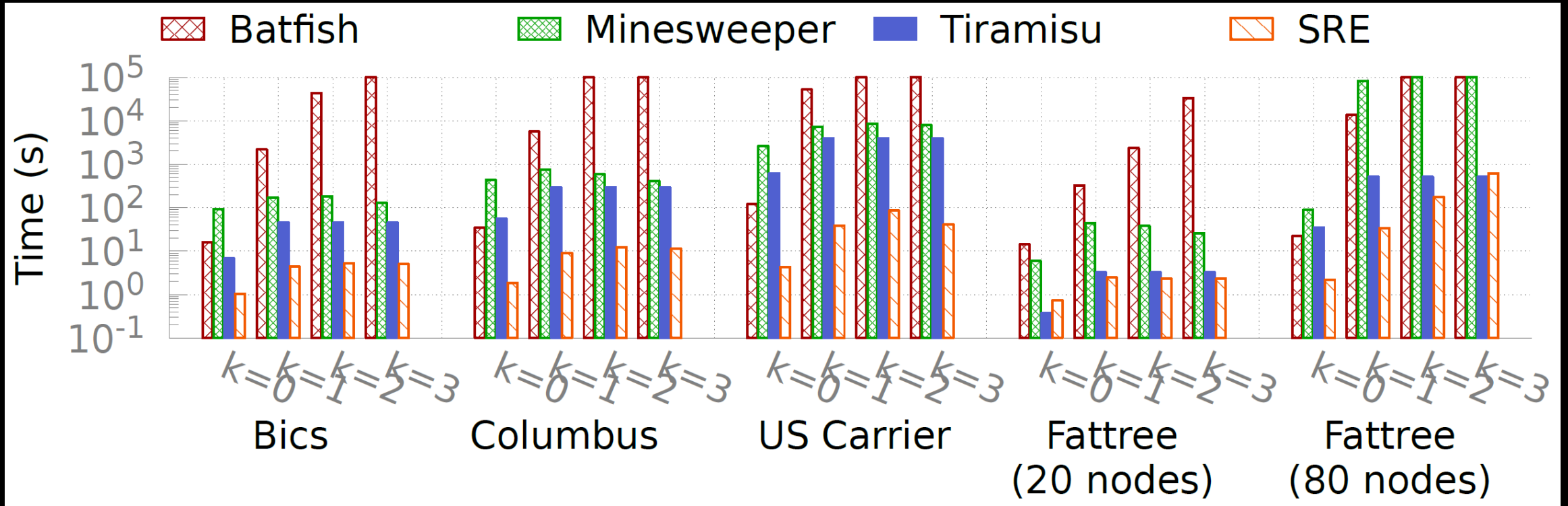
Experiment: compute all-pairs reachability



Experiment: compute all-pairs reachability



Experiment: compute all-pairs reachability



Scales to the product space
of headers and failures
by computing PFECs



SRE

Generalizes to deterministic and
probabilistic failures by delaying
binding to a failure model